

人工智能引论 26Spring

ELANOR

1 搜索——逻辑和CSP

1.1 什么是CSP?

CSP就是给变量赋值，让它们不违反约束的问题。

1.1.1 CSP三要素

变量

记作 $X_1, X_2, \dots, X_n$

它们就是“需要被赋值的東西”。

例子：

- 地图着色：每个区域是一个变量 SA, WA, NT...
- 数独：每个格子是一个变量
- 八皇后：第 i 行皇后的位置是变量 Q_i

域

每个变量可以取的可能值。

例子：

- 地图着色：{red, green, blue}
- 数独：{1,...,9}
- 八皇后：{1,...,N}

约束

规定哪些赋值组合是允许的、哪些是不允许的。

例子：

- 地图着色：相邻区域不能同色
- 数独：每行、每列都不能重复
- 八皇后：皇后不能攻击彼此

1.2 CSP v.s. SSP (Standard Search Problem)

1.2.1 结构化

黑盒状态 (普通搜索)

在迷宫里，一个状态可能是：

(robot_x = 7, robot_y = 12, path_history = [...], is_door_open = True, ...)

但算法对它一无所知，只能：

- 看它是不是终点
- 看它下一步能往哪走
(状态内部的信息不能被直接利用)

→ **黑盒**，无法分析结构。

结构化状态 (CSP)

比如“地图着色”：

```
WA = red
NT = green
SA = ?
Q = ?
```

这个状态有明确结构：

- 每个变量有固定域（颜色）
- 我们知道哪两个变量有约束（相邻）
- 我们知道哪些取值现在还合法
- 可以计算剩余可选值数量（MRV）
- 可以推断某些值立刻不可能（forward checking）

→ **状态内部规则清晰、可被利用，所以叫结构化。**

1.2.2 结构化的好处？

因为结构更清楚，我们就能用：

- MRV（最少剩余值）
- LCV（最少限制值）
- 前向检查
- 边一致性
- 回溯剪枝

这些“**专门为 CSP 优化的算法**”，而普通搜索用不了。

1.3 约束图

1.4 CSP的常见分类

1.4.1 离散变量

有限域

变量的取值来自一个有限集合，如布尔变量 {True, False}、地图颜色 {red, green, blue}、数独数字 {1...9}、八皇后列号 {1...N}

无穷域

字符串长度、全体实数等

1.4.2 连续变量

高铁到站时间等

常见处理方法：线性约束->线性规划LP

非线性约束复杂不表

1.5 CSP求解——回溯搜索

我们需要给所有变量赋值，而且要满足约束。

最简单的方式是：尝试一个变量 → 试一个值 → 检查是否违反约束 → 如果违反则回溯，否则继续 → 如果所有的值都失败，说明之前某个赋值导致无解，回溯并且换一个值。

1.5.1 优化——MRV (Minimum Remaining Value) 最小剩余值启发

先选择“可选值最少”的变量赋值

早点发现冲突

1.5.2 优化——LCV (Least Constraining Value)

选给其他变量带来最少限制的那个

更难走进死胡同

1.5.3 优化——FC (Forward Checking) 前向检查

提前删掉不可能值

当你给 X 赋一个值后，立即：

- 看它的邻居哪些值不可能了
- 提前从邻居的域里删掉不合法的值
- 如果某邻居的域变成空 → 立即回溯（不用继续了）

1.5.4 优化——约束传递 (AC Arc Consistency 边一致性)

Q1: 什么叫一条边 $X \rightarrow Y$ 是一致的 (consistent)?

对 X 的每一个可能取值，都存在至少一个 Y 的合法取值，不会违反约束。

如果存在某个 X 的取值 v:

- 无论 Y 取什么值都会冲突
→ 那么 v 就必须从 X 的域中删掉。

这就是 arc consistency 的核心操作。

Q2: 与前向检查的对比

- 前向检查只有在 某变量被赋值后 才会删别人的值
- Arc consistency 不需要等赋值
- 只要发现“不可能组合”就会删

因此能更早发现失败。

Q3:缺点

全局用AC太慢，发现不了3个一起的冲突

1.6 SAT (Boolean Satisfiability Problem, 布尔可满足性问题) 初步

给你一堆只包含True/False的逻辑公式，问有没有一种给变量赋True/False的方式，使得所有公式都成立？

1.6.1 命题逻辑的元素

常量 (True / False)

变量 (X, Y...)

连接符 ($\wedge$, $\vee$, $\neg$)

字符 literal 是一个常量、变量，或者它们的非。分为两种：正文字X，反文字 $\neg X$ 。

语句 sentence

世界 world 把对于所有变量的一个赋值组合称为一个世界

SAT即为：是否存在某个世界使所有语句都为真。

1.6.2 CSP与SAT转化的例子——N皇后

例子：N-皇后

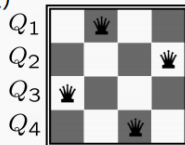


变量: Q_k (第k行的皇后所在的列数)

值域: $\{1, 2, 3, \dots, N\}$

约束: $\forall i, j \quad Q_i, Q_j$ 互相没有威胁

或者更明确点: $(Q_1, Q_2) \in \{(1, 3), (1, 4), \dots\}$
...



40

例子：N-皇后



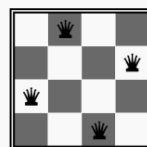
变量: X_{ij} (第i行, 第j列是否有皇后)

值域: $\{0, 1\}$

约束:

关于第i行: $(X_{i1} \wedge \neg X_{i2} \wedge \dots \neg X_{iN}) \vee (\neg X_{i1} \wedge X_{i2} \wedge \dots \neg X_{iN}) \dots$

关于第j列: $(X_{1j} \wedge \neg X_{2j} \wedge \dots \neg X_{Nj}) \vee (\neg X_{1j} \wedge X_{2j} \wedge \dots \neg X_{Nj}) \dots$



1.6.3 逻辑的互相转换

De Morgan's Law

$$\neg(A \wedge B) = (\neg A) \vee (\neg B)$$

$$\neg(A \vee B) = (\neg A) \wedge (\neg B)$$

分配律

$$A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$$

$$A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C) \star \star$$

1.6.4 CNF Conjunctive Normal Form 合取范式 (AND of ORs)

多个子句用 $\wedge$ 连接，每个子句内是 $\vee$

例如：

$$(X_1 \vee \neg X_2 \vee X_3) \wedge (\neg X_3 \vee \neg X_4)$$

1.6.5 DNF Disjunctive Normal Form 析取范式 (OR of ANDs)

多个“与”项，用 $\vee$ 连接

例如：

$$(X_1 \wedge \neg X_2 \wedge X_3) \vee (\neg X_3 \wedge \neg X_4)$$

例子：DNF to CNF (因为 SAT 求解器 (DPLL、CDCL) 只能处理 CNF)

$$\begin{aligned} & (X_1 \wedge \neg X_2) \vee (\neg X_3 \wedge X_4) \\ &= \neg(\neg(X_1 \wedge \neg X_2) \wedge \neg(\neg X_3 \wedge X_4)) \\ &= \neg((\neg X_1 \vee X_2) \wedge (X_3 \vee \neg X_4)) \\ &= \neg((\neg X_1 \wedge X_3) \vee (\neg X_1 \wedge \neg X_4) \vee (X_2 \wedge X_3) \vee (X_2 \wedge \neg X_4)) \\ &= (\neg(\neg X_1 \wedge X_3) \wedge \neg(\neg X_1 \wedge \neg X_4) \wedge \neg(X_2 \wedge X_3) \wedge \neg(X_2 \wedge \neg X_4)) \\ &= (X_1 \vee \neg X_3) \wedge (X_1 \vee X_4) \wedge (\neg X_2 \vee \neg X_3) \wedge (\neg X_2 \vee X_4) \end{aligned}$$

★ 核心原理只有一句话 (非常重要)

$$(A \wedge B) \vee (C \wedge D)$$

要变成：

$$(A \vee C) \wedge (A \vee D) \wedge (B \vee C) \wedge (B \vee D)$$

也就是 把前面那两个括号里的所有组合，两两“对撞”一次。

⚠ 这是分配律的递归展开，不是编出来的，是数学规则。



1.7 DPLL算法

深搜+单子句传播+回溯

★ 第 1 小点：在深搜过程中依次设置每个变量的值

比如你有变量：

p_1 、 p_2 、 p_3 、 p_4 ...

DPLL 就是：

1. 选一个变量 p_1
2. 尝试赋值 $p_1 = \text{True}$
3. 用这个赋值去检查所有子句
4. 如果在此基础上出现矛盾 $\rightarrow$ 回溯，让 $p_1 = \text{False}$

★ 第 2 小点：当某个子句为假的时候 $\rightarrow$ 回溯

例如子句：

$$(\neg p_1 \vee p_2)$$

如果：

- $p_1 = \text{True} \Rightarrow \neg p_1 = \text{False}$
- $p_2 = \text{False}$

那么整个子句就是 $\text{False} \rightarrow$ 无法满足 $\rightarrow$ 必须回溯。

PPT 的重点：

遇到“空子句”就回溯。

🔗 第 50 页重点：单子句传递 (unit propagation)

PPT 写：

单字符传递 (unit propagation)：当某个子句只剩下一个字符 literal，其他都为假时.....
赋值使它为真。

也就是说：

如果你看到：

$$(\neg p_3 \vee p_4)$$

又已知：

- $\neg p_3 = \text{False}$ (即 $p_3 = \text{True}$)
- 那么这个子句剩下唯一能是真的 literal 就是 p_4

$\rightarrow$ 所以 DPLL 必须强制赋值 $p_4 = \text{True}$

这叫 强制赋值 (forced assignment)

★ 单子句传播 (Boolean Constraint Propagation, BCP)

PPT 50 页写:

重复使用单字符传递, 直到无法使用为止。

意思是:

- 你赋值一个变量
- 这个赋值可能导致一些子句变成 single literal
- 单子句再逼迫另一个变量赋值
- 那个变量的赋值又可能让更多子句变成 single literal...

这些 **强制赋值连锁触发**, 就叫 BCP。

它极度重要, 因为:

🌟 BCP 能从一个赋值推导出大量隐含赋值, 极大地剪枝搜索树

SAT 求解器之所以快, 就是靠 BCP。

1.8 CDCL (Conflict-Driven Clause Learning) 矛盾指引的子句学习

遇到冲突时, 不是单纯回溯, 而是分析冲突原因 → 学习一个新子句 → 下次不再犯同样错误

关键点: 隐含图

🔥 用途 1: 定位冲突真正的“罪魁祸首”

当出现冲突 (例如 $C=True$ 和 $C=False$ 同时发生),

DPLL 只能:

- 迷茫后退
- 退错层
- 很慢

但 CDCL 用隐含图可以精确追踪冲突的来源:

```
A=True → B=True → C=True
(¬C) → C=False
```

你马上能看到:

- A 的选择引发了整个链条
- B、C 是被 A 的决定隐含推出来的

图让你准确知道:

是哪几个决定导致了冲突, 而不是所有决定。

1.8.1 冲突分析

画隐含图，然后cut

演示：

Conflict Driven Clause Learning

CDCL 算法

隐含图
(implication graph)
构建赋值的层级顺序，
由BCP推出的算为同一个层级
边上标记由哪个子句推出

假定我们有一个
从零开始的计数
器负责隐含图的
层数。手动赋值
的时候就+1，
BCP赋值的时候
维持不变。

蓝色代表假，
橙色代表真，
灰色代表还没赋值

北京大学
PEKING UNIVERSITY

62

CDCL 算法

将隐含图一分为二，所有的自行赋值的 (蓝色节点) 必须分在同一区，矛盾节点 (红色) 必须被分到另一区

可以从这个一分为二学到什么？

观察所有在非矛盾区，并且有边直接传到矛盾区的节点。

他们就是导致矛盾的原因。那他们不能被同时满足，所以学到一个新的子句。

(not x1, not x6, not x7)

但是已经存在C4

蓝色代表假，
橙色代表真，
灰色代表还没赋值

北京大学
PEKING UNIVERSITY

65

CDCL 算法

将隐含图一分为二，所有的自行赋值的 (蓝色节点) 必须分在同一区，矛盾节点 (红色) 必须被分到另一区

另一种分法？

学到
(not X1, not X6, not X5)

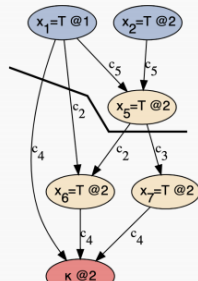
蓝色代表假，
橙色代表真，
灰色代表还没赋值

北京大学
PEKING UNIVERSITY

66

CDCL 算法

将隐含图一分为二，所有的自行赋值的（蓝色节点）必须分在同一区，矛盾节点（红色）必须被分到另一区



再一种?

学到
(not X1, not X5)

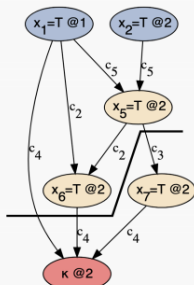
蓝色代表假,
橙色代表真,
灰色代表还没赋值



67

CDCL 算法：比较分法

将隐含图一分为二，所有的自行赋值的（蓝色节点）必须分在同一区，矛盾节点（红色）必须被分到另一区

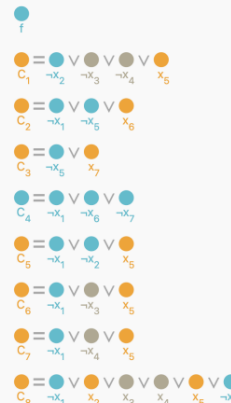


学到
(not X1, not X6, not X5)

我们的目的是触发越多的BCP越好

X1来自于第一层，X6来自于第二层，X5来自于第二层。首先我们就在第二层，并且已经触发矛盾，所以至少要回溯到第一层。但是回溯到第一层。那X1有值，X6和X5都还没有赋值，那利用这个新的子句，我们不能触发BCP。

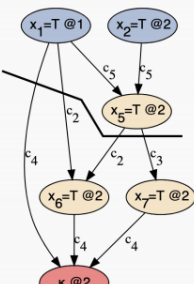
蓝色代表假,
橙色代表真,
灰色代表还没赋值



68

CDCL 算法：比较分法

将隐含图一分为二，所有的自行赋值的（蓝色节点）必须分在同一区，矛盾节点（红色）必须被分到另一区

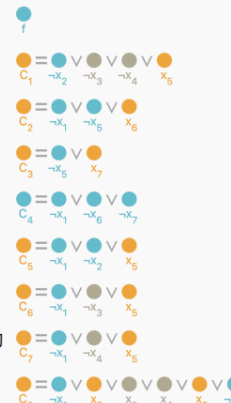


(not X1, not X5)

我们的目的是触发越多的BCP越好

X1的赋值来源于层级1，X5的赋值层级2。所以我们需要回溯到层级1，这样我们便可以马上BCP（原则：回溯到学到的子句里的second largest层级）

蓝色代表假,
橙色代表真,
灰色代表还没赋值



69

1.8.2 1-UIP cut (first unique implication point)

定义：在当前decision level中，所有路径从decision到冲突，第一次汇合的唯一节点，就是1-UIP
上例中，page69给出的cut是1-UIP cut。因为冲突发生在第二层级，所以需要在第二层级找，而x5是唯一在所有通往冲突路径上的DL2 共同节点（Unique Implication Point）。

并且注意，cut完之后要回溯到学到的子句里的second largest层级，在上例中也即是说，回溯到 x_1 中重新赋值，然后触发BCP。

2 搜索——对抗搜索

3 搜索——蒙特卡洛搜索

符号规定：

- k ：动作的数量（比如，有多少老虎机）
- $q_*(a)$ ：动作 a 的真实价值
- $Q_t(a)$ ：在时间 t 的时候，对动作 a 的估值
- $N_t(a)$ ：在时间 t 及之前，动作 a 被选中的次数
- R_t ：在时间 t 得到的价值
- A_t ：在时间 t 采取的动作

3.1 ϵ -greedy

做法：设定一个小概率 ϵ , $1-\epsilon$ 选目前最好的（利用）， ϵ 随机乱选一个（探索）

缺点：对所有非最优动作一视同仁，不够聪明。

3.1.1 固定值选择

通常会选一个很小的值，比如 **0.1** 或 **0.01**。PPT 第 25 页的那张折线图，它对比了三种选法：

- $\epsilon = 0$
(纯贪心)
：完全不探索。你可以看到绿线一开始很快，但很快就平了，因为它可能一辈子都没发现那台真正好的机器。
- $\epsilon = 0.1$
(蓝线)
：探索力度大。它能**最快**找到最优动作（曲线斜率最陡），但最后它选到最优动作的概率只能维持在 80% 多，因为它雷打不动地要花 10% 的时间去“乱试”。
- $\epsilon = 0.01$
(红线)
：探索力度小。它找最优动作的速度很慢，但由于它乱试的次数少，它的**上限**会比 0.1 更高。

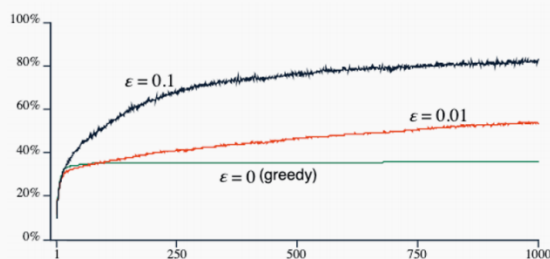
如何平衡不确定性: ϵ -greedy



- 三种不同的 ϵ

- $\epsilon = 0$
- $\epsilon = 0.1$
- $\epsilon = 0.01$

- 跑2000次实验, 每次实验会选择1000次动作, 选到最优动作的概率?



25

3.1.2 衰减策略

在很多高性能的 AI 算法里, 这个小概率不是固定的, 而是一个**从大到小**的过程:

- **初期:** 设其为1.0或0.5。这时候 AI 像个小孩子, 几乎全是随机乱尝试, 目的是为了快速见世面, 把所有可能性都摸一遍。
- **后期:** 随着尝试次数增加, 让它逐渐减小 (比如每走一步减一点, 直到减到 0.01 甚至 0)。
- **目的:** 先充分探索, 等心里有数了, 就收心专注于拿高分 (利用)。

3.2 UCB(Upper Confidence Bound)最大置信

公式: $A_{t+1} = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$ (argmax: 让数值达到最大的那个选项 (参数))

3.2.1 $Q_t(a)$ ——眼前的利益 (利用)

含义: 到目前为止, 动作 a 的平均收益。

逻辑: 如果这个动作之前总赢, 它的 Q 就高。这是“贪心”的部分, 它引导我们去选那些看起来已经成功的路径。

3.2.2 $c \sqrt{\frac{\ln t}{N_t(a)}}$ ——未知的潜力 (探索)

t (分子): 总共尝试的总次数。随着实验进行, t 越来越大, $\ln t$ 也会缓慢增长。

$N_t(a)$ (分母): 这个动作被选中的次数。

如果一个动作被选中的次数 N 很少, 分母就很小, 导致整个根号项变得巨大。

- 这就像是在对算法说: “嘿! 这个动作我们几乎没试过, 虽然它现在的平均分 Q 可能不高, 但它充满了不确定性, 我们要给它一个**奖金 (Bonus)** 去试一试!”

c (超参数): 这是调节系数。如果 c 很大, AI 就会变得非常“好奇”, 到处乱试; 如果 c 很小, AI 就会变得很“保守”, 只敢走老路。

3.3 MCTS四步循环

从UCB到博弈：选择 (selection)

我方执黑，现在对方（白棋）下完，轮到我方做决策

- 每个步骤中都采取UCB
- 相当于每一个节点都是一个老虎机
- 已经运行了大量模拟来积累所显示的统计信息
- 每个圆圈都包含
获胜次数 / 通过该节点次数
- 由于是博弈问题，每个节点的获胜都是针对自身的

这张PPT很重要！！

3.3.1 选择Selection

动作：从根节点（当前局面）开始，像走迷宫一样往下走。

规则：每到一个分叉路口，都算出所有子节点的 UCB 值，然后用 $\arg \max$ 选出最大的那个往下走。

注意（细节）：你会一直选，直到遇到一个“叶子节点”（即这个节点下面还有没试过的棋步）。

重点（PPT 36-37）：在选择过程中，立场是交替的。我方选 UCB 最大的，轮到对手选时，由于 UCB 反映的是该节点的胜率，对手也会选对他最有利（对我方最不利）的路径。

3.3.2 扩展Expansion

动作：当你选到一个还没被完全探索的节点时，增加一个新节点。

规则：从这个位置选一个还没试过的动作，创建一个子节点，初始化它的得分为 0/0。

目的：这就是“树长高/长宽”的过程。

3.3.3 模拟Simulation

动作：从刚才那个新节点开始，闭着眼睛乱走。

规则：不进行 UCB 计算，而是用最简单的随机策略（或者一些快速的启发式规则）一直下到游戏结束。

目的：看看在这种随机尝试下，最后是黑赢还是白赢。

进阶（PPT 42）：实际操作中，为了省时间，可能不会下到死，而是下几十步后用一个简单的分数来代替结局。

3.3.4 回溯Backpropagation

动作：拿着刚才模拟出来的结果（赢 = 1，输 = 0），顺着刚才走下来的路，原路返回。

规则：更新沿途所有节点的统计数据。每个节点的访问次数 N 都加 1。如果是赢了，获胜次数也加 1。

结果：路径上所有节点的 Q 值都会被刷新，这直接影响下一次“选择”阶段的 UCB 计算。

3.4 Alpha-Go的优化

3.4.1 选择优化

在标准的 UCB 公式里，如果一个动作没试过，我们的探索完全是盲目的。但 AlphaGo 引入了一个新变量： $P(s, a)$ （先验概率）。

AlphaGo 的选择公式变成了这样（简化版）：

$$A_{t+1} = \arg \max_a \left[Q_t(a) + \underbrace{u(s, a)}_{\text{新的探索项}} \right]$$

其中这个 $u(s, a)$ 里面包含了一个关键项： $P(s, a)/(1 + N_t(a))$ 。

那么 $P(s, a)$ 是什么？

- 含义：它是通过学习人类高手（职业棋手）的几百万局棋谱得来的。
- 直觉：这就像是一个拥有 20 年棋龄的高手在旁边指点：“喂，根据我的经验，这个局面下，下在‘天元’附近通常是好手。”
- 作用：

在算法刚开始、尝试次数 N 很少的时候，这个 $P(s, a)$ 占主导地位。结果：AI 不再“瞎试”，而是优先探索人类高手认为大概率正确的地方。

3.4.2 模拟优化

在普通 MCTS 中，第三步“模拟”是随机乱走。在围棋里，随机下棋的水平极低，算出来的胜率根本不准。

1. 更好的模拟：快速走子策略 (Rollout Policy) —— PPT 58/60

- 改进：AlphaGo 不再纯随机下棋，而是训练了一个“快速走子网络”
 - 特点：这个网络虽然不如主脑聪明，但它能在微秒内给出一个“像样”的招式。
 - 结果：模拟出来的这一局棋，质量比随机下棋高得多，胜率反馈也更准。

2. 引入“预言家”：价值网络 (Value Network) —— PPT 58

这是 AlphaGo 最伟大的发明之一。

- 黑科技：它训练了一个深层神经网络，只要给它看一眼现在的棋盘，它就能直接预测：“这个局面，黑棋有 70% 的概率会赢。”
- 作用：它不需要真的把棋下完（不需要跑模拟），就能直接给出一个分数。

3. 综合评估（混合打分制）—— PPT 58/59

AlphaGo 非常谨慎，它不完全相信“预言家”，也不完全相信“模拟”。它把两者的结果加权平均：

$$V(s_L) = (1 - \lambda) v_\theta(s_L) + \lambda z_L$$

v_θ ：预言家（价值网络）给的分。 z_L ：实战模拟（快速走子）最后赢没赢。

- 结果：这种双重保险让 AlphaGo 对每一个节点的评估都极其精准。

3.4.3 回溯优化

回溯：在第四步更新节点时，它不再是简单地+1或者+0，而是把上面算出来的那个综合评分 V 往上传。

输入信息：普通的 AI 可能只看棋盘黑白子，AlphaGo 的输入有 48 个通道 (48 channels)。它不仅看棋子，还看“气”有多少、这个子是什么时候下的、是否处于“征子”状态等高级特征 (PPT 60)。

3.5 AlphaGo Zero

不看人类棋谱，从零开始自我博弈。

4 机器学习基础和线性回归

4.1 机器学习模型分类

4.1.1 模型种类

1. 判别式模型 (Discriminative Models):
 - 核心逻辑: 给定输入 x , 预测标签 y 。
 - 数学本质: 学习条件概率 $P(y|x)$ 。
 - 通俗理解: 给它一张照片, 它告诉你这是猫还是狗。它在学习“分类边界”。
2. 描述式模型 (Descriptive Models):
 - 核心逻辑: 给定输入 x , 描述它出现的概率。
 - 数学本质: 学习概率分布 $P(x)$ 。
 - 通俗理解: 它在研究数据本身的规律, 比如“这类图片通常长什么样”。
3. 生成式模型 (Generative Models):
 - 核心逻辑: 给定一个隐变量 z (可以理解为灵感或随机种子), 生成一个新的数据 x 。
 - 数学本质: 学习 $P(x|z)$ 。
 - 通俗理解: 现在的 AI 绘图、ChatGPT 写的诗, 都属于这一类。

4.1.2 学习设置

1. 监督式学习 (Supervised Learning):
 - 特点: 有标准答案。我们需要提供标签 y
 - 例子: 给 1 万张标好“猫”或“狗”的照片让机器学。
2. 非监督式学习 (Unsupervised Learning):
 - 特点: 没有标准答案。不需要标签。
 - 例子: 给机器一堆照片, 它不知道是什么, 但它发现其中一堆颜色相近, 另一堆形状相近, 自动把它们聚成几类。
3. 强化学习 (Reinforcement Learning):
 - 特点: 通过与环境交互来获取数据。
 - 例子: 扫地机器人撞到墙 (惩罚), 扫干净了地 (奖励), 通过不断试错学会避障。

4.2 分类&回归

对于监督式学习而言, 我们要预言的“标签 y ”的取值范围决定了这是一个什么性质的任务。而分类与回归是最基础的两大类任务。

4.2.1 回归 regression

- 特点: 标签 y 是连续 (continuous) 的数值。
- 例子: 预测房子的价格 (比如 100.5 万、250.3 万)。

4.2.2 分类 classification

- 特点: 标签 y 是离散 (discrete) 的取值, 也就是“类别”。

- 例子：预测狗的品种（哈士奇、萨摩耶、阿拉斯加）。

4.3 模型训练过程Train

1. 准备训练数据 (Training Data):

- 我们有一堆带标签的例子，比如 (email1, not_spam), (email2, spam)...

2. 特征提取 (Feature Extraction):

- 机器看不懂感性的文字，它只认识数字。
- **核心细节**：我们需要把邮件转化成特征向量 x_i 。比如这封邮件里，“attention”出现了1次，“million”出现了2次.....这样，一封感性的邮件就变成了一串冷冰冰的数字 $[1; 2; 1; 0; 3; 1]^T$ 。
- 对应的标签 y_i 也会数字化，比如 $\{-1; 1\}$ 。

3. 模型训练 (Model Training):

- 我们将这些向量化的数据喂给算法。
- **目标**：寻找一个函数 f ，使得对于所有的 i ，预测值 $f(x_i)$ 都尽可能接近真实的标签 y_i 。即： $y_i \approx f(x_i); \forall i$ 。

4.4 模型评估

4.4.1 两种误差

1. 训练误差 (Training Error):

- **定义**：模型在它见过的“练习题”（训练集）上的平均误差。
- **目的**：我们通过**最小化**这个误差来训练模型。

2. 测试误差 (Test Error):

- **定义**：模型在它**没见过**的“新题”（测试集）上的平均误差。
- **目的**：这才是真正衡量模型好坏的标准，它反映了模型的**泛化 (generalization) 能力**——即模型举一反三的能力。

4.4.2 两种错误拟合

1. 过拟合 (Overfitting):

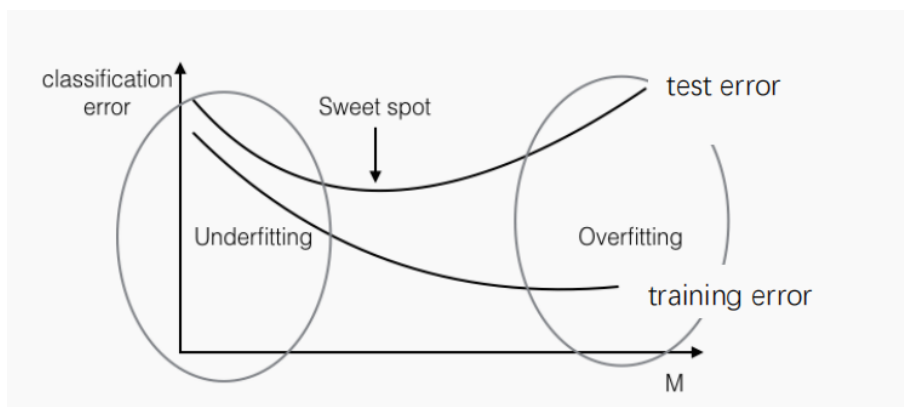
- **定义**：测试误差远大于训练误差。
- **通俗理解**：死记硬背。
- **PPT 上的经典例子**：
 - 如果我们给 10 封垃圾邮件 (Spam)，发现它们都包含“dollars”这个词。
 - 机器学到一个规则：**只要有“dollars”就是垃圾邮件**。
 - 在训练集上，它的表现是满分（训练误差为 0）。
 - 但到了真实的测试集，很多正常商务邮件也有“dollars”啊！于是机器疯狂报假警，这就是过拟合。

总结：它错把训练集中的**特殊规律**（偶然性）当成了**普遍规律**（必然性）。

2. 欠拟合 (Underfitting):

模型太笨，训练集都一塌糊涂。

4.4.3 Sweet spot



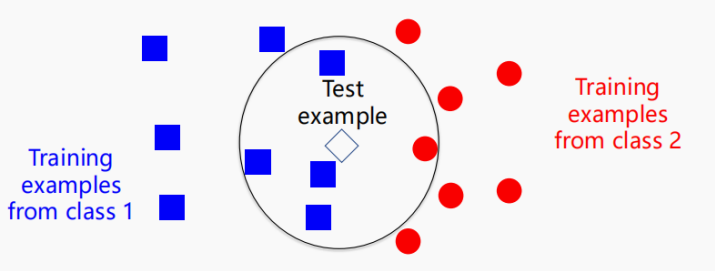
- **横轴 M**：代表模型的复杂度（比如模型越来越深、参数越来越多）。
- **纵轴 Error**：代表误差。
- **训练误差（实线）**：随着模型越来越复杂，它总能拟合得越来越好，所以训练误差是一路向下的。
- **测试误差（虚线）**：刚开始，随着模型变强，测试误差也减小。但到达一个临界点后，模型开始“走火入魔”去学那些无意义的干扰信息，于是测试误差反而开始反弹。
- **老师敲黑板**：中间那个测试误差最低的点，就是我们追求的 **Sweet spot**（甜点位）。在这个点之前是欠拟合，之后是过拟合。

4.5 k-NN算法 (k-Nearest Neighbor, k-近邻)

k近邻 (k-Nearest Neighbor, k-NN) 算法

 **北京大学**
PEKING UNIVERSITY

- 一个最简单机器学习算法
 - 对于一个测试样本，用训练样本中距离它最近的k个样本中占多数的标签来预测测试样本



1. 优点：

- **不需要训练**：它属于“懒惰学习”（Lazy Learning）。你给它数据，它直接存起来就完事了，不像其他算法需要花好几个小时去算参数。
- **原理极其简单**：只需要一个距离函数。
- **默认工具：欧氏距离 (Euclidean distance)**：

$$\circ \text{ 公式: } \|x_1 - x_2\| = \sqrt{\sum_{j \in [d]} (x_1^{(j)} - x_2^{(j)})^2}$$

2. 缺点：

- **存储成本高**：你必须把所有训练样本都带在身上，这就像去考试时不能把书背下来，必须把图书馆所有书都搬进考场。

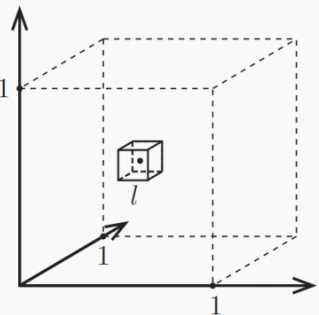
- **预测速度慢**：每预测一个新样本，都要把它和库里所有的样本比一遍距离，数据量大时会非常卡。
- **距离函数难找**：
 - 简单的几何距离不一定能反映真实的“相似度”。

4.6 维度灾难

维度灾难 (Curse of Dimensionality)



- 假设 $k=10$ ，总共有 n 个训练样本从 $[0,1]^d$ 中均匀采样



- 对某个测试样本，设 l 为能包含住其所有 k 近邻的超立方体的边长
- 则 $l^d \approx \frac{k}{n} \rightarrow l \approx \sqrt[d]{\frac{k}{n}}$
- 当 $d = 1000$ ，我们来计算 l

d	l
2	0.1
10	0.63
100	0.955
1000	0.9954

- 距离在高维空间失去意义，不再能很好地衡量远近
- k -NN 不适合高维空间

维度较高时，为了找到 n 个相邻的数据，你几乎要搜遍整个空间！

4.7 非参数化模型 & 参数化模型

维度灾难告诉我们，在高维世界里“查书”太难了；所以我们更倾向于**参数化模型**，把海量数据浓缩成几个核心参数，既省地方又跑得快。

1. 非参数化模型 (Non-parametric Models):

- **典型代表**： k -NN。
- **特点**：模型没法用几个固定的参数来描述。你需要**保留所有原始数据**。
- **比喻**：去考试，你是带了一整座图书馆（原始数据）进去翻书查。

2. 参数化模型 (Parametric Models):

- **典型代表**：线性回归、神经网络。
- **特点**：模型可以用有限个**可训练的参数**来定义。训练好之后，你就可以**丢弃原始训练数据**，只留下一套参数。
- **公式**： $y \approx f(x)$ ，这里的 f 就是由参数决定的模型。
- **比喻**：你把知识都学进脑子里了，变成了脑子里的神经元连接强度（参数）。考试时，你什么书都不用带，只靠脑子里的公式和逻辑就能解题。

4.8 线性模型 Linear Models、线性回归

4.8.1 线性模型 $f(x) = w^T x + b$.

输入 $x \in \mathbb{R}^d$ ：这就是我们的特征向量，比如一封邮件的词频，或者一个房子的各种属性。 d 代表维度。

权重 $w \in \mathbb{R}^d$ (Weight)：这是模型的核心参数。每一个维度对应一个权重。权重反映了该特征的重要性。如果某个特征的权重很大，说明它对最终结果影响很大；如果权重接近0，说明这个特征基本没用。

偏置 $b \in \mathbb{R}$ (Bias)：这也是一个可训练的参数。

输出 $f(x) \in \mathbb{R}$ ：模型给出的预测数值。

4.8.2 线性回归训练

1. 预测值：我们用模型算出来的结果 $f(\mathbf{x}_i)$
2. 真实值 (Groundtruth)：真实的标签 y_i
3. 平方损失函数 (Squared Loss):
 - 公式： $L(f(\mathbf{x}_i), y_i) = (f(\mathbf{x}_i) - y_i)^2$.
 - 逻辑：把预测和真实的差值平方。为什么平方？因为我们要消除正负号的影响，且平方会极大地惩罚那些离谱的离群点（差值越大，平方后的数值惩罚越重）。
4. 训练目标 (PPT 底部公式):
 - 我们在训练集上计算所有样本的平均损失。
 - 目标是找到一组参数 w, b 使得这个总损失最小。 $\min_{w, b} \frac{1}{n} \sum_{i \in [n]} L(f(\mathbf{x}_i), y_i)$
 - 这套方法也被称为最小二乘法 (Least Squares)。

4.8.3 梯度下降

你想下山，最稳妥的方法就是：用脚尖试探出最陡的那个下坡方向，顺着它走一小步，然后再重新判断。这样一步步走下去，最终就会到达谷底。

线性回归训练



- 如何对以下最优化问题求解？

$$\min_{w, b} \frac{1}{n} \sum_{i \in [n]} L(f(\mathbf{x}_i), y_i)$$

- 将要优化的目标 (objective) 写作变量 w, b 的函数

$$J(w, b) = \frac{1}{n} \sum_{i \in [n]} L(f(\mathbf{x}_i), y_i)$$

- 随机选定 w, b 的初始值，逐步调整 w, b 以降低 $J(w, b)$
- 每次沿着使 $J(w, b)$ 变小最快的方向使 w, b 走一小步

$$w \leftarrow w - \alpha \cdot \frac{\partial J(w, b)}{\partial w}, \quad b \leftarrow b - \alpha \cdot \frac{\partial J(w, b)}{\partial b}$$

- 梯度下降 (Gradient Decent)

33

梯度下降



$$w \leftarrow w - \alpha \cdot \frac{\partial J(w, b)}{\partial w}, \quad b \leftarrow b - \alpha \cdot \frac{\partial J(w, b)}{\partial b}$$

- $\frac{\partial J(w, b)}{\partial w} = \begin{bmatrix} \frac{\partial J(w, b)}{\partial w_1} \\ \vdots \\ \frac{\partial J(w, b)}{\partial w_d} \end{bmatrix} \in \mathbb{R}^d$, $\frac{\partial J(w, b)}{\partial b} \in \mathbb{R}$, 两者一同构成了 $J(w, b)$ 的梯度 (gradient)

$$\nabla J(w, b) = \begin{bmatrix} \frac{\partial J(w, b)}{\partial w} \\ \frac{\partial J(w, b)}{\partial b} \end{bmatrix} \in \mathbb{R}^{d+1}$$

- $\alpha \in \mathbb{R}$: 学习率 (learning rate), 是一个事先指定的超参数 (hyperparameter), 不随 w, b 一起优化
- 梯度下降：沿梯度的相反方向走一步，步长为 $\alpha \cdot \nabla J(w, b)$
- 相当于在每个维度上，沿着负偏导的方向移动一定距离，距离正比于偏导的绝对值，比例为 α

34

1. 定义误差项 e_i :

- $e_i = w^T x_i + b - y_i$
- 这个 e_i 就是模型预测值和真实值之间的差值（误差）。

2. 损失函数 $J(w, b)$:

- 它是所有样本误差平方的平均值: $\frac{1}{n} \sum (e_i)^2$ 。

3. 求偏导（求梯度）:

- 根据链式法则, $(e_i)^2$ 对 w 的导数是 $2e_i \cdot x_i$ 。
- 对 b 的导数是 $2e_i$ 。

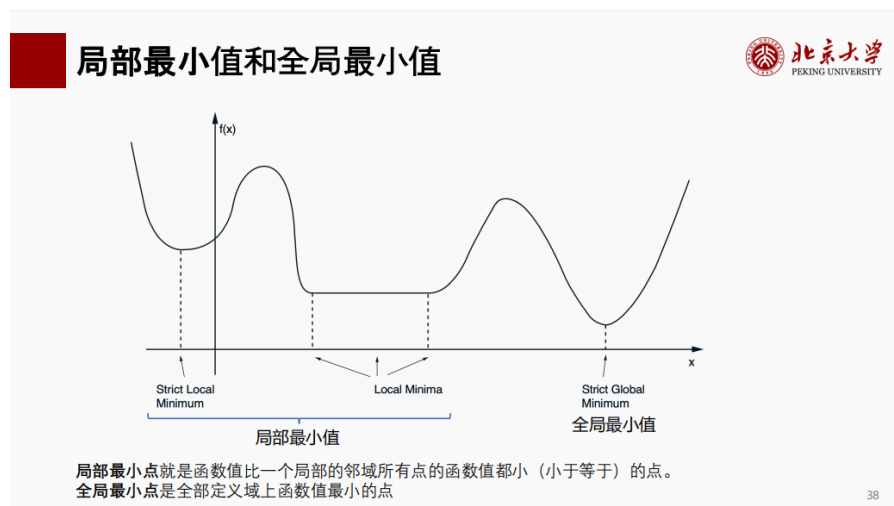
4. 最终的更新公式:

- $w \leftarrow w - \alpha \cdot \frac{2}{n} \sum e_i x_i$
- $b \leftarrow b - \alpha \cdot \frac{2}{n} \sum e_i$
- 老师敲黑板: 请看这两个公式。
 - 更新 w 时, 我们不仅参考了误差 e_i , 还乘上了特征 x_i 。这说明哪个维度对误差贡献大, 哪个维度的权重就改动大。
 - 更新 b 时, 直接看整体平均误差。

训练什么时候停止?

我们要一直迭代更新, 直到 $J(w, b)$ 几乎不再下降 (比如变动小于 $1e-4$), 或者达到了我们预设的步数。

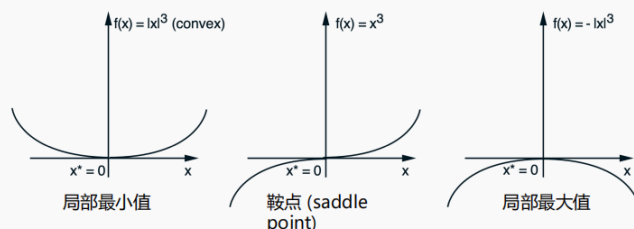
4.8.4 局部最小值



局部最小值和全局最小值



- 全局最小值也是局部最小值，但局部最小值不都是全局最小值
- 局部最小值的必要条件：
 - If x^* is a local minimum point of $f(x)$, then $\nabla f(x^*) = 0$
 - 为什么不是充分条件？



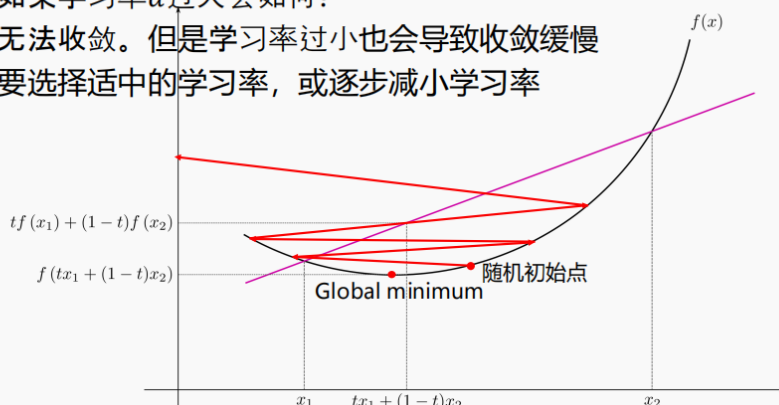
但是损失函数 $J(w; b) = \frac{1}{n} \sum (w^T x_i + b - y_i)^2$ 是关于 $[w^T, b]^T$ 的凸函数，因此一定可以采用梯度下降找到全局最小值。

4.8.5 学习率的调整

学习率的调整



- 如果学习率 α 过大会如何？
- 无法收敛。但是学习率过小也会导致收敛缓慢
- 要选择适中的学习率，或逐步减小学习率



4.8.6 线性回归流程总结

给定训练数据 $\{(x_i, y_i)\}$, 学习率 α 。

初始化模型参数 随机选取一个 w, b 。

训练模型

a. 计算模型预测值: $\hat{y}_i = w^T x_i + b, \forall i \in [n]$

b. 计算平方损失函数: $J(w, b) = \frac{1}{n} \sum_{i \in [n]} (\hat{y}_i - y_i)^2$

c. 计算梯度

$$\frac{\partial J(w, b)}{\partial w} = \frac{2}{n} \sum_{i \in [n]} (\hat{y}_i - y_i) x_i, \frac{\partial J(w, b)}{\partial b} = \frac{2}{n} \sum_{i \in [n]} (\hat{y}_i - y_i)$$

d. 梯度下降更新 w, b

$$w \leftarrow w - \alpha \cdot \frac{\partial J(w, b)}{\partial w}, b \leftarrow b - \alpha \cdot \frac{\partial J(w, b)}{\partial b}$$

e. 重复以上步骤，直到损失函数不再下降或达到预设迭代次数

5 逻辑回归、多分类与正则化

5.1 逻辑回归

5.1.1 经验风险最小化框架 ERM

确定采用的模型 $f(x)$ -> 确定损失函数 $L(f(x), y)$ -> 把平均损失最小化 (e.g. 利用梯度下降优化参数) 大部分监督学习算法都遵循以上经验风险最小化框架 (ERM)，区别仅在于具体选择的 f 和 $L(f, x, y)$

5.1.2 二分类问题

二分类 (Binary Classification)

- 标签只有两种
 - $y \in \{-1, 1\}$, -1代表负类 (negative class), 1代表正类 (positive class)
 - 如垃圾邮件识别, 1代表垃圾邮件, -1代表正常邮件
- 一般不直接让 $f(x) \in \mathbb{R}$ 拟合 $y \in \{-1, 1\}$
- 为了将实数输出转换为类别 $\{-1, 1\}$, 采用 $\text{sign}()$ 函数
 - $\text{sign}(f(x)) = \begin{cases} 1, & \text{if } f(x) > 0 \\ -1, & \text{if } f(x) < 0 \end{cases}$
- 使用什么损失函数?
 - 最直接的目标, 最小化分类错误数, 使用零一损失函数 (zero-one loss)
 - $L(f(x), y) = \begin{cases} 0, & \text{if } \text{sign}(f(x)) = y \\ 1, & \text{if } \text{sign}(f(x)) \neq y \end{cases} \Leftrightarrow L(f(x), y) = \begin{cases} 0, & \text{if } y \cdot f(x) \geq 0 \\ 1, & \text{if } y \cdot f(x) < 0 \end{cases}$ ($f(x) = 0$ 时默认0损失)
 - 但是, 零一损失函数是阶跃函数, 不可微 (non-differentiable) 且不连续 (non-continuous), 无法用梯度下降优化 (在0处不可微, 其余处梯度都为0, 无法提供下降方向)

zero-one loss

5.1.3 逻辑回归是什么

处理二分类问题, $f(x) = w^T x + b$, 交叉熵损失函数

5.1.4 最大似然估计MLE

为什么不用0-1损失? 不可微且不连续, 不可用梯度下降来优化。

一个理解: 这里为什么要弄最大似然估计? 等价于最小化交叉熵损失

MLE最大似然估计

- 通过最大化观测数据在给定概率模型下的似然 (例如: 把训练样本预测为正确标签的概率) 来估计模型参数
 - 如果训练样本互相独立 (独立同分布假设), 则最大似然估计可写为
$$\max_{\theta} \prod_{i \in [n]} p(y_i | x_i; \theta), \text{ 或简写为 } \max_{\theta} \prod_{i \in [n]} p(y_i | x_i; \theta)$$
 - 但是, 大量概率连乘容易造成数值超出计算精度, 例如 $0.5^{1000} \approx 9 \times 10^{-302}$
 - 解决方法为, 最大化对数似然 (log-likelihood)
$$\max_{\theta} \log \left(\prod_{i \in [n]} p(y_i | x_i; \theta) \right) \Leftrightarrow \max_{\theta} \sum_{i \in [n]} \log(p(y_i | x_i; \theta))$$

7

注意区分概率和似然:

概率: 参数 θ 固定, 问某个结果发生的概率 $p(\text{数据} | \theta)$

似然 (likelihood): 数据固定 (就是你手上的训练集), 把它当成 θ 的函数

$L(\theta) = p(\text{数据} | \theta)$ 。所以“观测数据在给定概率模型下的似然”就是这件事:

$$L(\theta) = p(\{(x_i, y_i)\}_{i=1}^n | \theta)$$

最大化似然是什么意思？

就是在选 θ ，让上面这个乘积尽量大：

$$\max_{\theta} \prod_{i=1}^n p(y_i | x_i; \theta)$$

直觉版：对每个样本，模型都应该给“真实标签”一个尽量高的概率，只要有些样本真实标签概率很低，乘积就会被拖到很小 $\Rightarrow$ 这组参数会被淘汰

一个例子

最大似然估计例子

- 给定一枚正反不均匀的硬币
- 已知抛了 n 次硬币，其中正面朝上的次数为 m 次
- 用最大似然估计 (MLE) 估算硬币正面概率
- 解法：
 - 首先对抛硬币事件进行概率建模，假设每次抛硬币正面朝上的概率为 p ，则反面为 $1 - p$
 - 在这个概率模型中， p 是我们唯一要估算的参数
 - 将观测数据在给定概率模型下的似然写出（假设观测数据为“正反正正正正正正正正正正...”，其中正面 m 次，反面 $n - m$ 次）：
$$Likelihood = p^m (1 - p)^{n - m}$$
 - 最大化对数似然来估计 p

$$\max_p m \log p + (n - m) \log(1 - p)$$
 - 让梯度为 0，得到 $\frac{m}{p} - \frac{n - m}{1 - p} = 0$ ，求得 $p = m/n$





北京大学
PEKING UNIVERSITY

Q：“参数是 p ”是什么意思？ p 不是概率吗？

A：参数 (parameter) = 模型里未知、需要从数据中估计的量。概率也可以是参数。

5.1.5 逻辑回归的最大似然估计 & 损失函数（交叉熵损失）

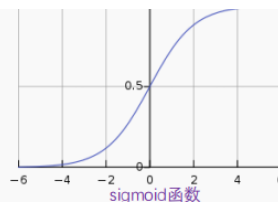
由于线性打分函数 $f(x) = w^T x + b$ 可以输出任何实数，而二分类问题里我们关心的是概率，所以引入 sigmoid 函数将实数压缩到 $[0, 1]$ 之间的实数。

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

关键推导：

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$1 - \sigma(x) = \frac{e^{-x}}{1 + e^{-x}} = \frac{1}{1 + e^x} = \sigma(-x)$$



- 则 $p(y = 1 | x; \theta) = p(y = 1 | x; w, b) = \sigma(f(x)) = \sigma(y \cdot f(x))$
- 自然的， $p(y = -1 | x; w, b) = 1 - \sigma(f(x)) = \sigma(-f(x)) = \sigma(y \cdot f(x))$
- 说明，不论 y 取 $1/-1$ ，都有 $p(y | x; w, b) = \sigma(y \cdot f(x))!$

- 在训练集上最大化对数似然

$$\max_{w,b} \sum_{i \in [n]} \log[p(y_i|x_i; w, b)] = \sum_{i \in [n]} \log[\sigma(y_i \cdot f(x_i))]$$

- 代入 $f(x_i) = w^T x_i + b$, 得到最优化问题

$$\begin{aligned} \max_{w,b} \sum_{i \in [n]} \log[\sigma(y_i(w^T x_i + b))] &= \sum_{i \in [n]} \log\left[\frac{1}{1+e^{-y_i(w^T x_i + b)}}\right] \\ &= -\sum_{i \in [n]} \log[1 + e^{-y_i(w^T x_i + b)}] \end{aligned}$$

- 写成最小化平均损失函数的形式 (逻辑回归的ERM形式) :

$$\min_{w,b} \frac{1}{n} \sum_{i \in [n]} \log[1 + e^{-y_i(w^T x_i + b)}]$$

- 由最大对数似然推出的损失函数 $L(f(x_i), y_i) = \log[1 + e^{-y_i(w^T x_i + b)}]$ 称为交叉熵损失 (Cross Entropy Loss), 有时也直接称为 Logistic/Log Loss

- 使用梯度下降求解如下最优化问题

$$\min_{w,b} \frac{1}{n} \sum_{i \in [n]} \log[1 + e^{-y_i(w^T x_i + b)}]$$

- 对目标 $J(w, b) = \frac{1}{n} \sum_{i \in [n]} \log[1 + e^{-y_i(w^T x_i + b)}]$ 求梯度

$$\frac{\partial J(w,b)}{\partial w} = -\frac{1}{n} \sum_{i \in [n]} \frac{e^{-y_i(w^T x_i + b)}}{1+e^{-y_i(w^T x_i + b)}} y_i x_i = -\frac{1}{n} \sum_{i \in [n]} [1 - p(y_i|x_i; w, b)] y_i x_i$$

$$\frac{\partial J(w,b)}{\partial b} = -\frac{1}{n} \sum_{i \in [n]} \frac{e^{-y_i(w^T x_i + b)}}{1+e^{-y_i(w^T x_i + b)}} y_i = -\frac{1}{n} \sum_{i \in [n]} [1 - p(y_i|x_i; w, b)] y_i$$

- 把样本 i 预测为真实标签的概率越接近1 (说明已经充分拟合该样本), 它对梯度的贡献越小

- 迭代更新 w, b 直到 $J(w, b)$ 无法再下降或达到预设的最大次数

$$w \leftarrow w - \alpha \cdot \frac{\partial J(w,b)}{\partial w}, \quad b \leftarrow b - \alpha \cdot \frac{\partial J(w,b)}{\partial b}$$

12

	线性回归	逻辑回归
任务	回归	二分类
模型	线性模型 $f(x) = w^T x + b$	线性模型 $f(x) = w^T x + b$
输出	$f(x)$	$\text{sign}(f(x))$ 直接输出 $\{-1, 1\}$; 或 $\sigma(f(x))$ 输出取1的概率
损失函数	平方损失 $(f(x_i) - y_i)^2$	交叉熵损失 $\log[1 + e^{-y_i(w^T x_i + b)}]$

5.2 多分类问题

5.2.1 OvR的做法

对K分类问题训练K个二分类器。

问题: 训练成本高; K个二分类器互相独立, 模型不统一

5.2.2 Softmax回归

$$p(y = k|x) = \frac{e^{f_k(x)}}{\sum_{j=1}^K e^{f_j(x)}}$$

它保证: 每个概率都在 (0, 1); 总和一定是1; 强调指数的放大效应 (某个 f_k 明显更大时, $p(y = k|x)$ 会非常接近 1)

- 使用最大对数似然估计参数

- 假设 $f_k(x)$ 的参数为 θ_k , 最大化训练集 $\{(x_i, y_i) | i \in [n]\}$ 的对数似然:

$$\max_{\{\theta_k\}} \sum_{i \in [n]} \log[p(y_i | x_i)] = \sum_{i \in [n]} \log \left[\frac{e^{f_{y_i}(x_i)}}{\sum_{j \in [K]} e^{f_j(x_i)}} \right]$$

- 等价于最小化交叉熵损失:

$$\min_{\{\theta_k\}} -\frac{1}{n} \sum_{i \in [n]} \log \left[\frac{e^{f_{y_i}(x_i)}}{\sum_{j \in [K]} e^{f_j(x_i)}} \right] = \frac{1}{n} \sum_{i \in [n]} (\log[\sum_{j \in [K]} e^{f_j(x_i)}] - f_{y_i}(x_i))$$

5.3 正则化

$$\min_f \frac{1}{n} \sum_{i=1}^n L(f(x_i), y_i) + \lambda \cdot R(f)$$

$\lambda \cdot R(f)$ 称为正则化项, 用于惩罚过于复杂的模型

$\lambda > 0$ 是超参数, 预先指定, 不随参数优化

5.3.1 正则化在惩罚什么?

1. 少数特征维度支配预测 (某些 w_j 变得特别大) -> L2 正则化
2. 很多没用特征仍被赋予非零权重 (模型把噪声也当成了信号) -> L1 正则化

5.3.2 L1 正则与 L2 正则

L1 正则:

$$R(w) = \|w\|_1 = \sum_{j=1}^d |w_j|$$

它更倾向让很多维度的权重变成 0 (鼓励稀疏)

L2 正则:

$$R(w) = \|w\|_2^2 = w^T w = \sum_{j=1}^n w_j^2$$

平方会“放大大权重”, 用于惩罚少数过大的权重维度

- 岭回归 (Ridge Regression)

- 线性回归 + L2 正则化

$$\min_{w,b} \frac{1}{n} \sum_{i \in [n]} (w^T x_i + b - y_i)^2 + \lambda \|w\|^2$$

- 梯度

$$\frac{\partial J(w,b)}{\partial w} = \frac{2}{n} \sum_{i \in [n]} (w^T x_i + b - y_i) x_i + 2\lambda w \in \mathbb{R}^d$$

- Lasso 回归

- 线性回归 + L1 正则化

$$\min_{w,b} \frac{1}{n} \sum_{i \in [n]} (w^T x_i + b - y_i)^2 + \lambda \|w\|_1$$

- 支持向量机 (Support Vector Machine, SVM) (最大间隔准则)

- 合页损失 (Hinge Loss) + L2 正则化

$$\min_{w,b} \frac{1}{n} \sum_{i \in [n]} \max(0, 1 - y_i(w^T x_i + b)) + \lambda \|w\|^2$$

5.3.3 为什么正则化只出现 ω 而通常不正则 b ?

b 只是整体平移决策边界，不代表某个特征的复杂度

5.3.4 判断 λ 怎么选

训练损失小 & 测试损失大 $\rightarrow$ 过拟合 $\rightarrow$ 调大 λ

训练损失和测试损失都大 $\rightarrow$ 欠拟合 $\rightarrow$ 调小 λ

5.3.5 K-折交叉验证

- 有时总数据量比较少，10%的验证集不足以准确地验证模型性能
- K-折交叉验证：
 - 将测试集外的所有数据随机分为 K 份
 - 对一组给定的超参数组合：
 - 每次使用 K-1 份数据训练模型，用 1 份验证模型误差
 - 将 K 次验证的模型误差取平均来衡量该组超参数的好坏
 - 遍历所有可能的超参数组合
 - 返回平均误差最低的超参数组合，在测试集上最终测试模型性能

K 折相当于“每个样本都当过验证集一次”，评价更稳、更不依赖一次随机划分。

6 决策树与随机森林

6.1 决策树

6.1.1 为什么要学决策树?

之前学的线性分类器本质上是 $f(x) = w^T x + b$ 打分，然后按正负号分类。它对应的分类边界是一个超平面（只有两个特征的时候就是一条直线），将整个空间“一刀切”划分为两部分。

希望设计出具有比 $\text{sign}(w^T x + b)$ 逻辑更复杂的分类模型

6.1.2 决策树是什么

- **决策树**是一种常见的机器学习方法
- 定义：**决策树**是一种树形结构，包含一个**根节点**、若干个**内部节点**和若干个**叶节点**。其中：
 - **根节点**包含样本全集
 - 每个**非叶节点**对应于一个（特征）属性测试；其包含的样本集合根据属性测试的结果被划分到子节点中
 - 每个**叶节点**对应于决策结果（取多数类）

训练决策树时，我们希望递归地选择一个属性，使得划分后每个子集都尽量只包含同类标签。

能做到这一点的属性，就是好的划分属性。

比如PPT筛选科学家的例子里，“是否喜欢吃香蕉”这个节点的筛选没有使得纯度变高，所以是一个坏划分。

6.1.3 纯度度量与划分准则

如何度量一个集合的纯度？

- 信息熵(一个随机变量的不确定性)
 - 熵：对于离散型随机变量 X ，服从概率分布 $p(x)$ ，其信息熵定义为

$$H(x) = -\sum p(x) \log_2 p(x) = \sum p(x) \log_2 \frac{1}{p(x)}$$

• 定义理解

- 对于某一事件 $X = x$ ，其发生的概率 $p(x)$ 越小，则包含的“信息量”越大
- 当 $X = x$ 为一确定事件，即 $p(x) = 1$ 时，其所包含的“信息量”最小 (=0)
- 因此可以用 $\log_2 \frac{1}{p(x)}$ 来描述该事件所包含的“信息量”
- 信息熵 $H(X)$ 为系统内不同事件(X 取不同值)的“信息量”的期望值 $\mathbb{E}_x[\log_2 \frac{1}{p(x)}]$

性质1: $H(X) \geq 0$ 恒成立。当且仅当确定分布时等号成立（即存在一个确定事件概率为1，其他事件概率为0）

性质2: 分布越平均，熵越大 ($\log_2 x$ 为上凸函数，由 Jensen 不等式)

- 联合熵：如果我同时观察 (X, Y) ，它到底会落在哪个组合上这件事有多难猜

$$H(X, Y) = -\sum \sum p(x, y) \log_2 p(x, y)$$

- 条件熵：在已知随机变量 X 的条件下，随机变量 Y 还有多少不确定性。

$$H(Y|X) = \sum p(x) H(Y|X=x) = -\sum \sum p(x, y) \log_2 p(y|x)$$

- 联合熵和条件熵的关系：

$$H(X, Y) = H(Y) + H(X|Y) = H(X) + H(Y|X)$$

- 互信息：X和Y之间共享了多少信息

$$I(X, Y) = H(X) + H(Y) - H(X, Y)$$

知道 X 以后，能帮我减少多少关于 Y 的不确定性？

$$I(X, Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

- $I(X, Y) = I(Y, X)$
- $I(X, X) = H(X)$
- X, Y 相关性越强， $I(X, Y)$ 越大
- X, Y 相互独立时 ($p(x, y) = p(x)p(y)$), $I(X, Y) = 0$

对第二个式子的理解: $H(X|X)=0$

- 基尼指数

$$\text{Gini}(D) = \sum_{k \in [K]} \frac{|D_k|}{|D|} \left(1 - \frac{|D_k|}{|D|}\right) = 1 - \sum_{k \in [K]} \left(\frac{|D_k|}{|D|}\right)^2 \approx \sum_{k \in [K]} P(y=k)(1 - (P(y=k)))$$

$|D_k|$: 标签为第 k 类的样本数

基尼指数反映了从集合里随机抽取两个样本，它们类别不一样的概率。

如何选择一个好的划分准则？

- 信息增益

- 对于一个样本集合，把“标签随机变量”的熵，直接拿来当这个节点的混乱程度：

$$H(D) = -\sum \frac{|D_k|}{|D|} \log \frac{|D_k|}{|D|}$$

$|D|$ 是集合 D 的样本总数， D_k 是标签为第 k 类的样本子集 $\frac{|D_k|}{|D|}$ 就是经验分布下“标签等于 k ”的概率。

- 熵越低，节点越纯。当一个节点里全是同一类时，熵就是0
- 属性 A 对集合 D 的信息增益为：

$$g(D, A) = H(D) - \sum \frac{|D_{A=a_i}|}{|D|} H(D_{A=a_i})$$

$H(D)$ 代表划分前这个节点本身有多乱，第二项代表按属性 A 划分后各个子节点熵的加权平均。也就是划分后的平均混乱程度。

- 事实上，第二项就是标签相对于属性的条件熵 $H(Y|X)$ 。所以信息增益其实就是标签 Y 和属性 X 的互信息。

$$g(D, A) = I(Y, X)$$

一个属性的信息增益大，等价于说这个属性和标签之间共享的信息多。信息增益 = 按某个属性划分后，标签的不确定性减少了多少。

- 缺点：信息增益偏爱取值很多的属性。分得极纯但是不具有泛化性。

- 增益率：信息增益 ÷ 属性自身的不确定性

- $g_R(D, A) = \frac{g(D, A)}{H_A(D)}$ ，其中 $H_A(D) = -\sum_{i \in [m]} \frac{|D_{A=a_i}|}{|D|} \log \frac{|D_{A=a_i}|}{|D|}$

先看 $H_A(D)$

在 전체 9 个样本里：

- $A = \text{Yes}$ 有 4 个
- $A = \text{No}$ 有 5 个

所以

$$H_A(D) = -\frac{4}{9} \log_2 \frac{4}{9} - \frac{5}{9} \log_2 \frac{5}{9}$$

这个算的是 A 取 Yes/No 的分布熵。

再看 $H(D_{A=\text{Yes}})$

先把 $A = \text{Yes}$ 的样本挑出来。PPT 第 7 页这个子集是：

$$\{1, 2, 3, 8\}$$

这些样本的标签是：

$$\{+, +, -, -\}$$

所以在这个子集里：

- 正类 2 个
- 负类 2 个

于是

$$H(D_{A=\text{Yes}}) = -\frac{2}{4} \log_2 \frac{2}{4} - \frac{2}{4} \log_2 \frac{2}{4} = 1$$

这个算的是“认真工作的人”这一支里，标签  不乱。 11、决策树与随机森林 - 2025

- 这个 $H_A(D)$ 不是子集标签的熵，而是把属性 A 自己当成“随机变量”来算它的熵。

- 缺点：可能对可取值数目较少的属性有偏好

- （属性的）基尼指数

- 属性A对集合D的基尼指数：

$$\text{Gini}(D, A) = \sum_{i \in [m]} \frac{|D_{A=a_i}|}{|D|} \text{Gini}(D_{A=a_i})$$

- 其中，属性A的可能取值为 $\{a_i | i \in [m]\}$ ， $D^{A=a_i}$ 表示集合D中样本属性A取 a_i 的子集
- 选择基尼指数最小的

6.1.4 属性的离散到连续——二分法

对连续属性 A ，找一个阈值 t ，把样本分成两部分： $A > t$ 和 $A \leq t$ 。这样连续属性就被转成了一个二值离散属性。

候选划分点

假设连续属性在训练集上的不同取值升序排列为

$$a_1, a_2, \dots, a_m$$

那么候选划分点取相邻取值的中点：

$$T_A = \left\{ \frac{a_i + a_{i+1}}{2} \mid i = 1, \dots, m-1 \right\}$$

为什么取中点？

因为真正影响划分结果的，不是阈值具体是多少，而是它把样本分成了哪两边。只要阈值落在两个相邻取值之间，分法其实都一样，只是取中点最方便。

实际划分点

对于每个候选属性和每个候选划分点，都计算划分后的信息增益

$$g(D, A, t) = H(D) - \frac{|D_{A,t,+}|}{|D|} H(D_{A,t,+}) - \frac{|D_{A,t,-}|}{|D|} H(D_{A,t,-})$$

然后选使这个值最大的 (A, t)

不仅要比较哪个属性好，对连续属性还要比较这个属性用哪个阈值切最好

6.2 回归树

6.2.1 标签从离散到连续

“是否是一个好的科学家” $\rightarrow$ “是一个研究水平有多高的科学家”

分类变成线性回归打分。

以前看“纯不纯”是看类别混不混；现在标签变成连续数值后，就改成看这些数值散不散。

6.2.2 回归树

通过L2 Loss重新定义集合D的纯度

某个节点的预测值写成：

$$\bar{y}_D = \frac{1}{|D|} \sum_{j \in D} y_j, \text{ 即平均值}$$

再定义这个节点的损失为

$$L(D) = \sum_{j \in D} (y_j - \bar{y}_D)^2$$

如果按属性 A 划分，那么划分后的损失是

$$L(D, A) = \sum_{i \in [m]} L(D_{A=a_i})$$

然后选使它最小的属性： $A^* = \arg \min_{A \in F} L(D, A)$

Algorithm 1 How to build a decision tree

Input: Dataset D , Feature set $\mathcal{F}$

Output: Root Node $T = \text{BUILD_DECISION_TREE}(D, \mathcal{F})$

1: **function** BUILD_DECISION_TREE($D, \mathcal{F}$)

2: **if** $\mathcal{F} = \emptyset$ Or $|D| = 1$ **then**

3: $S \leftarrow \text{PURITY_SCORE}(D)$

4: **return** Leaf_Node $N = \text{MAKE_NODE}(D, S, \text{NULL})$

5: **end if**

6: $S_{\text{best}} \leftarrow \text{NULL}, A_{\text{best}} \leftarrow \text{NULL}$

7: **for** $A \in \mathcal{F}$ **do**

8: $S = \text{PURITY_SCORE}(D, A)$

9: $(S_{\text{best}}, A_{\text{best}}) \leftarrow \text{BETTER_SCORE}(S_{\text{best}}, S, A_{\text{best}}, A)$

10: **end for**

11: $\text{Array_D} \leftarrow \text{PARTITION_DATASET}(D, A_{\text{best}})$

12: Node $N = \text{MAKE_NODE}(D, S_{\text{best}}, A_{\text{best}})$

13: **for** $i = 0 \rightarrow |\text{Array_D}|$ **do**

14: $T \leftarrow \text{BUILD_DECISION_TREE}(\text{Array_D}[i], \mathcal{F} - A_{\text{best}})$

15: Node $N = \text{ADD_CHILD}(N, T)$

16: **end for**

17: **return** N

18: **end function**

19:

20: **function** PURITY_SCORE(D, A)

21: **return** Corresponding Purity_Score formula with D and A (omited if NULL)

22: **end function**

终止条件：属性集 F 为空，或只剩一个样本
也可以用一些提前终止条件，如深度达到某一上限、或纯度分数好于某一阈值

遍历所有当前属性集中的属性，计算纯度分数，选择分数最好的属性 A

按选择的属性划分样本集 D ，在每个子集 $D^{A=a_i}$ 上递归使用 $F - A$ 构建决策树

6.3 集成学习与随机森林

6.3.1 集成学习

集成学习 = 多个个体学习器（e.g. 线性模型、学习器...）+ 结合策略

“好而不同，各有所长”

6.3.2 随机森林——集成学习的例子

个体学习器：决策树

结合策略：样本扰动+属性扰动

- 样本扰动：对于每一棵决策树，先对训练集进行随机采样，得到一个独立的训练集。于是不同树看到的数据就不完全一样，所以他们会长得不一樣。
- 属性扰动：对于每一棵决策树，只选择数据集中的一部分特征进行子树划分训练。

最后，用所有训练好的决策树的平均预测（如多数类）作为对测试样本的输出

7 神经网络与反向传播矩阵

能否自动提取特征？

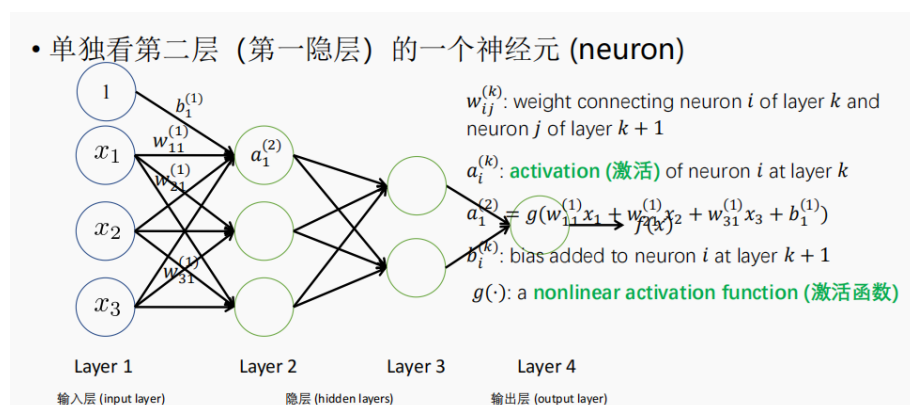
7.1 多层感知器MLP

也叫“全连接神经网络”。

- (1) **多层** 说明不再是“输入直接到输出”，而是中间加入了若干层隐藏层。
- (2) **全连接** 说明相邻两层之间，通常是“前一层每个神经元都连到后一层每个神经元”。
- (3) **宽度与深度** 宽度：某一层神经元的个数；深度：网络层数

多层感知器能表达比单层更复杂的分类边界，双隐层感知器足以解决任何复杂的分类问题。

7.2 正向传播



公式是：

$$a_1^{(2)} = g(w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + w_{31}^{(1)}x_3 + b_1^{(2)})$$

其中， $w_{ij}^{(l)}$ ：第 l 层第 i 个神经元，连到第 $l+1$ 层第 j 个神经元的**权重**

$a_j^{(l)}$ ：第 l 层第 j 个神经元的**激活值**（然后再作为下一轮的input）

$b_j^{(l)}$ ：加到下一层该神经元上的**偏置**（给模型一个“整体平移”的自由度）

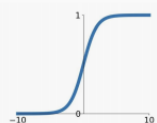
$g(\cdot)$ ：**非线性**激活函数

常见激活函数



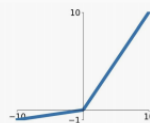
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



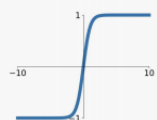
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

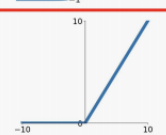


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

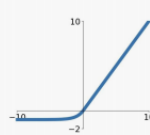
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

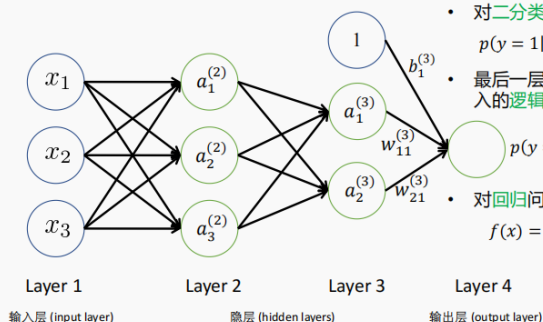


ReLU是很好的默认选择！在大部分常见问题上表现良好！

神经元计算 = 线性变换 + 非线性激活

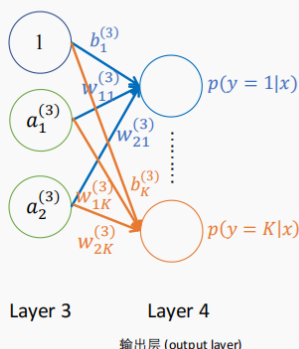
对于输出，不同任务的输出层写法取决于任务类型。

• 单独看输出层的一个神经元



- 对二分类问题，用sigmoid函数输出正类概率
 $p(y=1|x) = \sigma(w_{11}^{(3)} a_1^{(3)} + w_{21}^{(3)} a_2^{(3)} + b_1^{(3)})$
- 最后一层等价于一个以倒数第二层的激活作为输入的逻辑回归
- 对回归问题，直接输出
 $f(x) = w_{11}^{(3)} a_1^{(3)} + w_{21}^{(3)} a_2^{(3)} + b_1^{(3)}$

• 对多分类问题



- 对 K 分类，输出层使用 K 个神经元
- 使用softmax函数，第 k 个输出层神经元输出

$$p(y=k|x) = \frac{\exp(w_{1k}^{(3)} a_1^{(3)} + w_{2k}^{(3)} a_2^{(3)} + b_k^{(3)})}{\sum_{j \in [K]} \exp(w_{1j}^{(3)} a_1^{(3)} + w_{2j}^{(3)} a_2^{(3)} + b_j^{(3)})}$$
- 即，最后一层等价于一个softmax回归

7.3 神经网络训练

用反向传播求梯度（按计算图反向传梯度），进而用梯度下降训练所有参数 $\{w_{ij}^{(l)}, b_j^{(l)}\}$ 。

核心： $z = f(x), y = g(x)$, 则 $\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$.

下游梯度 = 上游梯度 * 本地梯度

sigmoid的导数

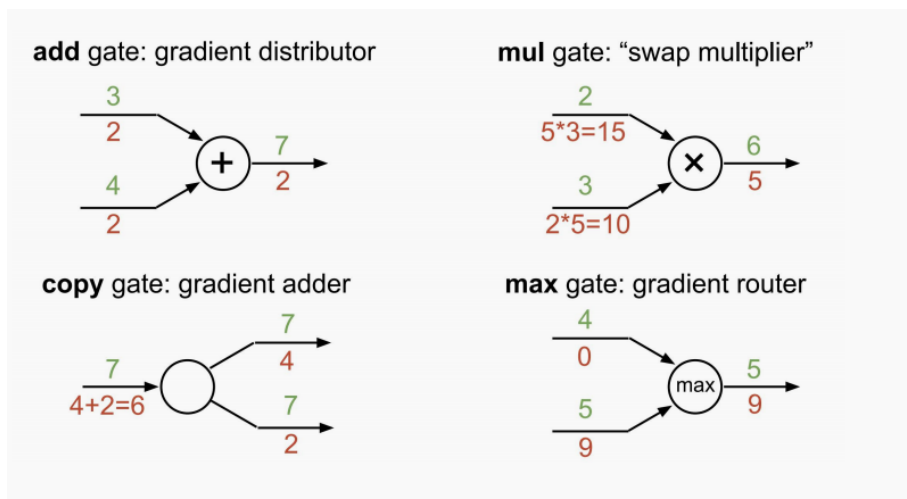
$\sigma(x) = \frac{1}{1+e^{-x}}$, 则它的导数是： $\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$.

因此如果是线性+sigmoid激活, $f = \sigma(s), s = x_0w_0 + x_1w_1 + w_2$,

$\frac{\partial f}{\partial s} = f(1-f)$, $\frac{\partial f}{\partial x_0} = f(1-f)w_0$, $\frac{\partial f}{\partial x_1} = f(1-f)w_1$.

但是由于sigmoid函数的导数最大也只有 $\frac{1}{4}$, 往前传的时候越乘越小, 可能导致梯度消失现象。而ReLU就不会有这种情况 (但是如果 $x \leq 0$ 也会出事)。

从节点传到模块化反传



加法节点对应梯度传1, 乘法节点乘另一个输入, sigmoid节点乘上 $\sigma(x)(1 - \sigma(x))$, ReLU节点正区间传1负区间传0.

7.4 矩阵形式

前向传播的矩阵形式

通用标量形式 (第L层输入维度为 d_L): $z_i^{(L+1)} = \sum_{j=1}^{d_L} w_{ji}^{(L)} a_j^{(L)} + b_i^{(L)}, a_i^{(L+1)} = g(z_i^{(L+1)})$

通用矩阵形式: $z^{(l+1)} = W^{(l)} a^{(l)} + b^{(l)}$, $a^{(l+1)} = g(z^{(l+1)})$

其中, $z^{(l+1)} \in \mathbb{R}^{d_{l+1}}$: 第 $l+1$ 层所有线性输出组成的列向量,

$W^{(l)} \in \mathbb{R}^{d_{l+1} \times d_l}$: 从第 l 层到第 $l+1$ 层的权重矩阵

$a^{(l)} \in \mathbb{R}^{d_l}$: 第 l 层所有神经元激活值组成的列向量

$b^{(l)} \in \mathbb{R}^{d_{l+1}}$: 第 $l+1$ 层的偏置向量

$g(\cdot)$: 逐元素作用的激活函数 (element-wise)

反向传播的矩阵形式

$\delta^{(l+1)} := \frac{\partial L}{\partial z^{(l+1)}} = \frac{\partial L}{\partial a^{(l+1)}} \odot g'(z^{(l+1)})$, 其中 $\odot$ 是逐元素乘法, 每个分量之间互不混合。

//穿过激活层, 得到传到线性层输出 z 的梯度。

$\frac{\partial L}{\partial W^{(l)}} = \frac{\partial L}{\partial z^{(l+1)}} (a^{(l)})^T$ // 算权重梯度

$\frac{\partial L}{\partial b^{(l)}} = \frac{\partial L}{\partial z^{(l+1)}}$ // 算偏置梯度

$\frac{\partial L}{\partial a^{(l)}} = (W^{(l)})^T \delta^{(l+1)}$ // 把梯度传回前一层

7.5 全梯度下降、随机梯度下降与小批量梯度下降

8 计算机视觉——图像分类与卷积神经网络

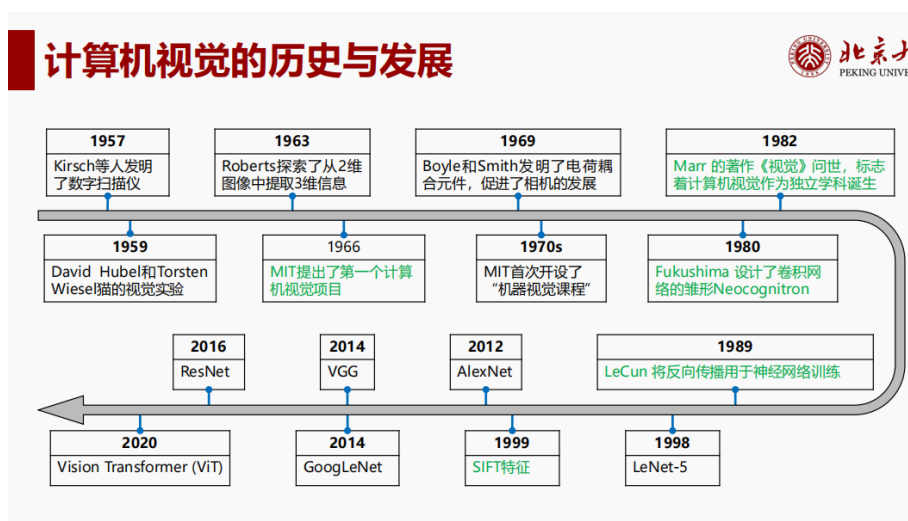
8.1 基础概念

视觉 = 从获取光信号, 到产生对现实世界理解的过程。

视感觉 把光信号变成可处理的信号

视知觉 在已有图像信息上进行理解

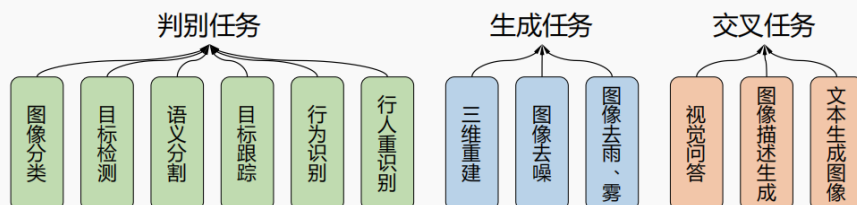
历史发展



计算机视觉的研究方向

计算机视觉的研究内容

- 计算机视觉的主要核心问题可分为**判别式**(discriminative)和**生成式**(generative)
- 另外，计算机视觉还与自然语言处理相互交融，衍生了一系列的**视觉-文本交叉任务**。



8.2 图像分类

图像分类 判断图像所属的类别标签

难点

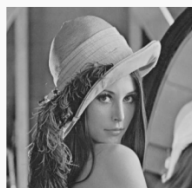
- 视角变化大:** 同一物体从不同角度看，外观差异很大
 - 姿态变化多:** 人或动物姿势不同，特征变化明显
 - 遮挡严重:** 物体可能部分被遮挡
 - 类内差异大:** 同一类别内部差异大，例如不同品种狗
 - 尺度变化大:** 同一物体在图中大小差异很大
- 总结一句话: **同一类物体的外观不一致，而不同类物体可能相似，这是图像分类的挑战。**

计算机中图像的表达

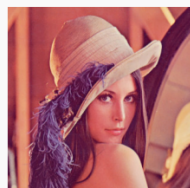
- 图像可以用一个定义在平面坐标上的二维函数 $I = f(x, y)$ 来表示
 - x 和 y 是平面**坐标**， f 为该点处的**亮度或灰度**，每个点称为一个**像素**
- 当 x, y 和 f 均为离散值时，称该图像为**数字图像**
 - 二值图像: 每个像素的亮度取自0和1，只有黑白两种颜色。
 - 灰度图像: 每个像素取0(黑)到255(白)之间的整数，表示不同的灰度级。
 - 彩色图像: 每个像素由RGB三通道分量构成，每个通道分别由各自灰度级描述。



二值图像



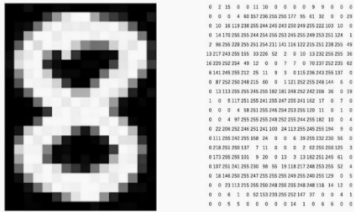
灰度图像



彩色图像

计算机中图像的存储

- 在计算机中，数字图像以数组的形式存储，数组的元素与图像的像素一一对应
- 存储每个像素需要的bit数量称为图像的**位深度**，bit是计算机存储的最小单元
- 图像的位数通常是8位，可以表示 $2^8 = 256$ 种数量级，灰度范围对应0-255



灰度图像及对应数组



计算机如何实现图像分类

计算机模拟神经元的活动，利用算法对图像数据进行处理，提取特征，通过分类器输出类别预测结果。

8.2.1 特征

特征空间 指特征向量（与线代无关!）所在的空间，每一个特征向量对应特征空间中一个点

比如对水果分类，重量作x轴，糖分作y轴，每个样本对应一个点

分类器 对数据进行分类的模型，最终输出分类结果

特征提取算子——梯度直方图HOG

希望用梯度方向分布描述局部目标的形状，因为物体的形状可以由局部边缘和梯度模式很好地表示，并且这种方法对光照和小幅度几何变形有不变性（用一些奇怪的方法保证的）。

• 步骤

- 1.灰度化
- 2.计算垂直和水平梯度，以及梯度幅值和方向

$$\text{幅值 } G = \sqrt{G_x^2 + G_y^2}, \text{方向 } \theta = \arctan \frac{G_y}{G_x}.$$
- 幅值代表强度（有多明显），方向代表哪个方向的变化最强
- 3.将图像划分成若干cell
- 4.统计每个cell内的梯度方向，分bin，投票（权重=梯度幅值）
- 5.Block归一化，相邻的 4 个 cell 组成一个 block，对 block 内所有 cell 的向量做 L2 归一化
目的：减少光照或对比度变化影响
- 6.拼接特征向量

最后拼接特征向量是什么意思啊？我没搞懂这是为了干什么

明白啦，Elanor，我帮你把 **HOG 的“拼接特征向量”**这个步骤彻底梳理清楚，用通俗的语言和实际意义解释。

📁 13、计算机视觉-图像分类与卷积神经网络 - 2025(1)

1 背景：为什么要拼接

- HOG 的特征是 **局部统计的直方图**：每个 **cell** → 一个长度 = bin 数的向量
- 但是一张图通常有很多 cell，每个 cell 只描述自己那块区域
- 我们希望让 **分类器看到整张图的完整特征**

所以要把所有 cell/block 的向量 **串起来**，形成一个大向量 → 描述整张图

2 拼接示意

假设：

- 一张图被划成 4 个 block，每个 block 4 个 cell
- 每个 cell → 3 个 bin → 3 个数字
- 每个 block → 4 个 cell → $3 \times 4 = 12$ 个数字

步骤：

1. 对每个 block 做 **L2 归一化**
2. 把所有 block 的向量 **按顺序拼接**
 - Block1 → [b1,b2,...,b12]
 - Block2 → [b13,...,b24]
 - Block3 → [b25,...,b36]
 - Block4 → [b37,...,b48]



最终 HOG 特征向量 = [b1, b2, ..., b48]

- 7.可视化

特征提取算子——Haar

特征提取算子——SIFT

8.2.2 分类器

KNN

2 相似度计算

- 常用度量：

1. **L1 距离**（曼哈顿距离）：

$$d_1(I_1, I_2) = \sum |I_1 - I_2|$$

2. **L2 距离**（欧几里得距离）：

$$d_2(I_1, I_2) = \sqrt{\sum (I_1 - I_2)^2}$$

- 例子（PPT 98 页）：两个 4×4 灰度图 → 计算每个像素差值的平方和 → 得到距离

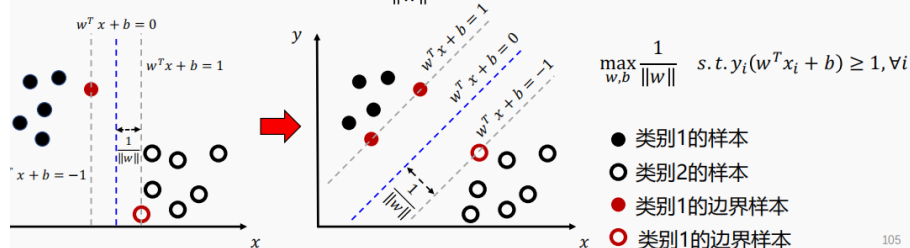
线性分类法

支撑向量机SVM

找到一个分界面，在正确分类的基础上，令样本到分界面的最小距离最大化

假设存在两个类别，标签 y 分别为1和-1。存在 $w^T x + b = 0$ 为两类的正确分界面，且 $w^T x + b = 1$ 和 $w^T x + b = -1$ 恰好将两类分开，即任意样 x_i 本均满足 $y_i(w^T x_i + b) \geq 1$ ，则所有样本到 $w^T x + b = 0$ 的最小距离为 $\frac{1}{\|w\|}$ 。

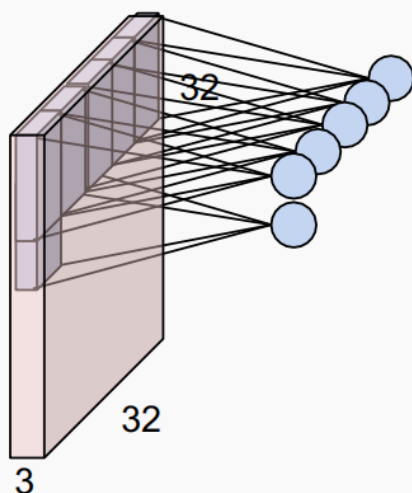
支撑向量机的目标为找到最大的 $\frac{1}{\|w\|}$



8.2.3 基于卷积神经网络CNN的图像分类算法

1. 卷积层 用很多个小窗口，在图像上滑动，提取局部特征

卷积层 (Convolution Layer)



比如这里输入图像是 $32 \times 32 \times 3$ ，意思是宽32，高32，有RGB 3个通道，所以它是一个厚度为3的长方体。

滤波器kernel是一个可学习的小模板，如 $5 \times 5 \times 3$ （最后一个数字必须和图像通道数一致），它一次看图像上对应大小的一个小块，然后把这个小块里的像素和滤波器里的参数逐个相乘求和取偏置，得到一个数。这个数表示：这个位置有没有出现这个滤波器关心的特征。

上图右边的小圆点代表，输入图像中的一个局部区域被链接到右边的一个输出神经元。

那滤波器怎么设计出来的？

滤波器一般不是人工设计出来的，而是 CNN 在训练过程中学出来的。

一开始，滤波器里的数值通常是随机初始化的，比如一个 $5 \times 5 \times 3$ 滤波器里面有 75 个小参数。

训练时流程是：

1. 用当前滤波器做卷积，得到分类结果。
2. 和真实标签比较，算损失。
3. 反向传播计算“滤波器里的每个数该怎么改，才能让损失变小”。
4. 用梯度下降更新滤波器参数。
5. 重复很多轮。

所以滤波器本质上就是一组可学习参数。

类比一下：

刚开始滤波器像一副乱配的眼镜，什么也看不清；训练很多次后，它慢慢变成能检测边缘、颜色、纹理、局部形状的小探测器。

补一句考试常用表达：

卷积层中的滤波器参数通过反向传播和梯度下降从训练数据中自动学习得到，而不是手工指定。

补零 zero padding 在刚才的例子中，最后输出的是 28×28 的激活图，我们希望输出尺寸一致，因此需要补零。

补零 (Zero padding)



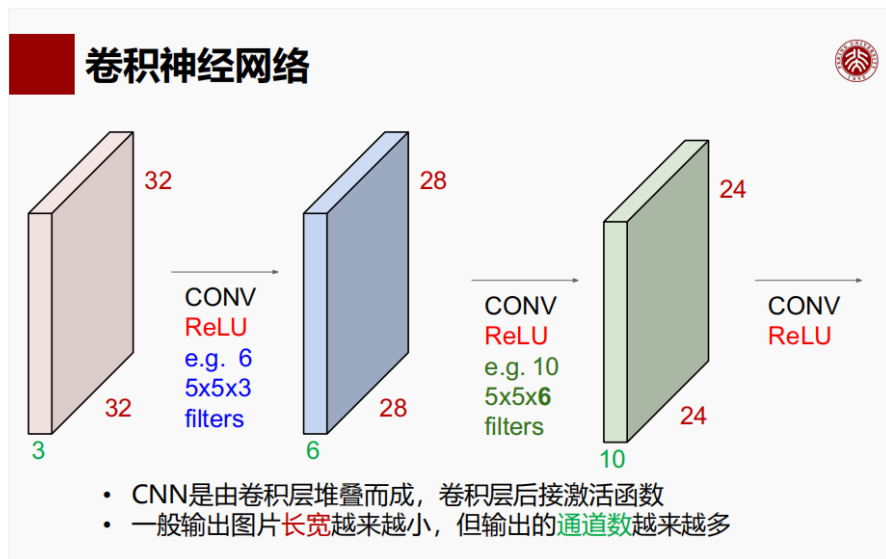
0	0	0	0	0	0			
0								
0								
0								
0								

- 考虑一个输入为 7×7 的图片，
- 滤波器大小为 3×3 ，padding with 1 个像素 =>
- **输出仍为 7×7 !**
- 通常如果滤波器大小为 $F \times F$ ，padding with $(F-1)/2$ ，就能保持原有特征图大小。

例如：

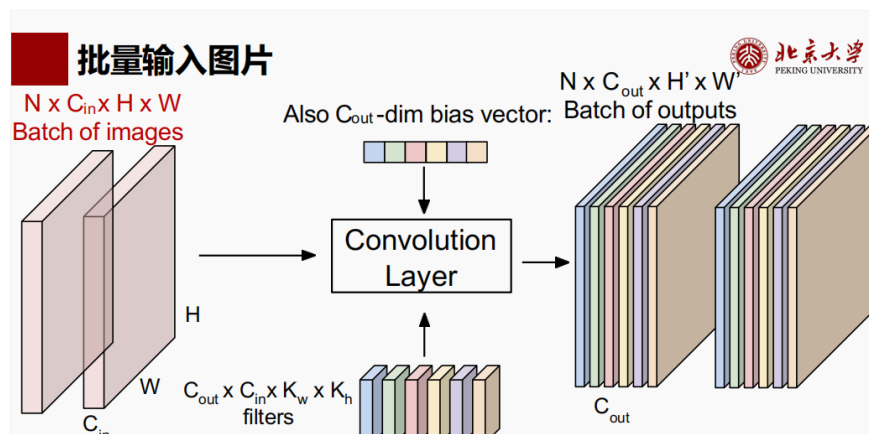
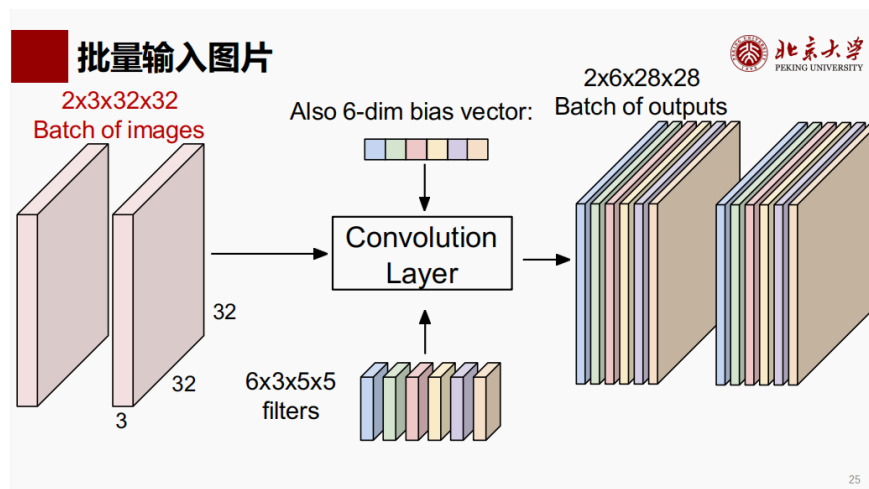
- $F = 3 \Rightarrow$ zero pad with 1
- $F = 5 \Rightarrow$ zero pad with 2
- $F = 7 \Rightarrow$ zero pad with 3

如果滤波器边长是偶数，则不能对称补零了。比如 $F=2$, $(F-1)/2=0.5$, 那就左边补一个右边补零个这样子。



不同的滤波器堆叠起来，每一层产生一个输出通道，6个就得到6个输出通道，所以下一层是 $28 \times 28 \times 6$

最终输出



2.池化层 把每个通道的特征图变小。

最大池化MaxPooling

规则固定，没有可学习参数；对小范围平移不敏感。

$$\begin{bmatrix} 1 & 1 & 2 & 4 \\ 5 & 6 & 7 & 8 \\ 3 & 2 & 1 & 0 \\ 1 & 2 & 3 & 4 \end{bmatrix}$$

用 2×2 的窗口，stride=2，每次看一个 2×2 小块，取最大值：

左上：

$$\begin{bmatrix} 1 & 1 \\ 5 & 6 \end{bmatrix} \Rightarrow 6$$

右上：

$$\begin{bmatrix} 2 & 4 \\ 7 & 8 \end{bmatrix} \Rightarrow 8$$

左下：

$$\begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix} \Rightarrow 3$$

右下：

$$\begin{bmatrix} 1 & 0 \\ 3 & 4 \end{bmatrix} \Rightarrow 4$$

所以输出是：

$$\begin{bmatrix} 6 & 8 \\ 3 & 4 \end{bmatrix}$$

直觉上，最大池化是在问：



这一小块区域里，这种特征有没有强烈出现？有的话保留最强的那个。

3.批归一化 batch normalization BN

把一批数据中每个特征维度标准化成均值约为0，方差约为1，让训练更稳定、更容易优化。

但在 CNN 里，卷积层输出不是简单向量，而是：

$$N \times C \times H \times W$$

这里最重要的“特征维度”通常就是 **通道 C**。

比如卷积层输出：

$$32 \times 64 \times 28 \times 28$$

意思是：32 张图，每张图有 64 张特征图，每张特征图大小 28×28 。

BN 会对**每个通道**分别算均值和方差。

也就是说，对第 1 个通道，把这 32 张图上所有 28×28 个位置的数拿出来，算一个均值和方差；第 2 个通道再算一套；一直到第 64 个通道。

所以它不是“把整张图所有数混在一起归一化”，而是：

每个通道单独标准化。

直觉上：

某个通道可能表示“竖直边缘强度”；

另一个通道可能表示“红色纹理强度”；

BN 就是让每一种特征的数值尺度更稳定，不要有的通道特别大、有的特别小。

也就是说，每个样本有多少维特征就要看它卷积层输出的通道数

BN层-训练阶段



Input: $x : N \times D$

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

可学习的权重和偏移量:

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\gamma, \beta : D$$

当 $\gamma = \sigma$ $\beta = \mu$ 时, 输出等于输入

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

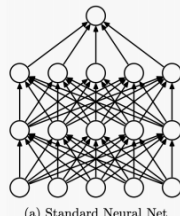
$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j \quad \text{最终输出, 形状为 } N \times D$$

4.Dropout 防止过拟合

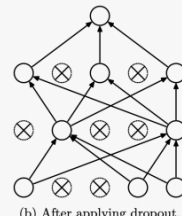
Dropout



- 当特征很多而样本不足时, 模型容易过拟合。
- 模型对微小扰动需要具有鲁棒性。
- Dropout:
 - 在训练过程的每一次迭代中, 以p的概率随机丢弃一些神经元。
 - 保留的神经元需要除以1-p, 以保持一层的期望不变。
- $$h' = \begin{cases} 0 & \text{概率为 } p \\ \frac{h}{1-p} & \text{其他情况} \end{cases}$$
- 在测试过程, 使用全部的神经元。



(a) Standard Neural Net



(b) After applying dropout.

5.全连接层 FC layer

前面一直在做特征提取, 得到一个三维特征 $H \times W \times D$

这里H是特征图的高, W是特征图的宽, D是通道数 (也就是有多少张特征图)。每个位置上的数表示某种特征在这个位置的激活强度。

全连接层要先 **拉直** flatten

排列成一个长一维向量, 然后用一个权重矩阵做线性变换 $y = Wx + b$ 。其中W是一个权重矩阵。输出一个向量是对不同类别打的分数。之后通常还会接softmax, 把这些得分变成概率。

这里是在说：全连接层怎么把前面提取到的特征，变成“每个类别的分数”。

假设 CNN 前面已经提取完特征，并拉直成：

$$x \in \mathbb{R}^{9216}$$

现在要分 10 类，比如：

猫、狗、车、马.....

全连接层会用一个权重矩阵：

$$W \in \mathbb{R}^{10 \times 9216}$$

再加一个偏置：

$$b \in \mathbb{R}^{10}$$

于是：

$$y = Wx + b$$

输出：

$$y \in \mathbb{R}^{10}$$

也就是 10 个数。

这 10 个数不是概率，而是类别得分 **class scores**。例如：

$$y = [2.1, -0.5, 3.7, 0.2, \dots]$$

第几个数最大，就说明模型当前最倾向于预测成第几个类别。



如果是 ImageNet，有 1000 个类别，那最后输出就是：

$$u \in \mathbb{R}^{1000}$$

9 计算机视觉——三维重建

9.1 从三维到二维：摄像机几何

9.1.1 针孔模型 pinhole model

假设相机前面有一个极小的小孔，光线只能通过这个小孔进入，然后投到后面的成像平面上。

结构大概是：

物体 → 小孔 → 成像平面

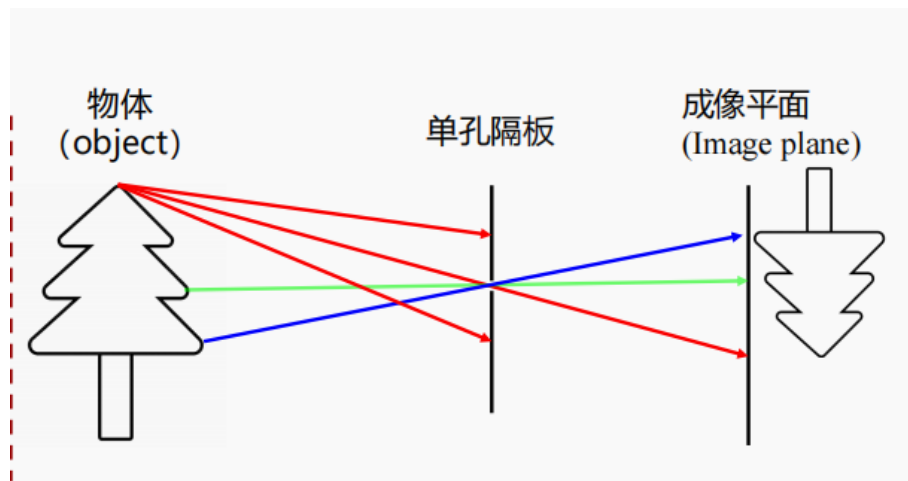
为什么需要这个小孔？

因为光是沿直线传播的。如果没有限制，一个物体上的每个点都会向各个方向反射光，成像平面上的每个位置都会收到来自很多空间点的混合光，图像就糊成一片。

小孔的作用就是：

只允许某一条特定方向的光线通过，从而让空间中的一个点对应到图像平面上的一个点。

这就是“投影”的物理基础。

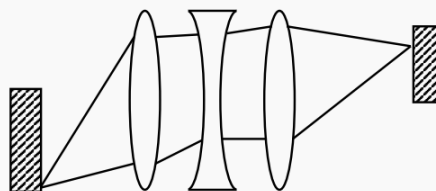


实际上小孔通光量太小，难以保证成像细节。真实相机一般用凸透镜组合，把空间中一个点发出的光汇聚到成像平面上的一点。

针孔模型

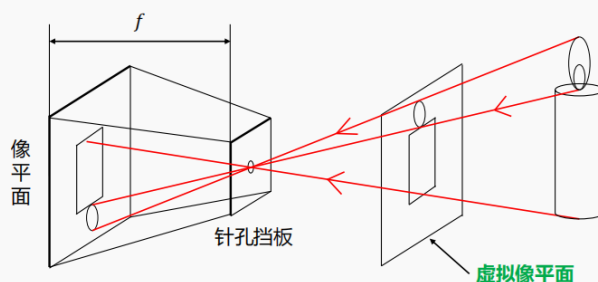


透镜组合的作用是将空间中一个点发出的光汇集到成像平面上一点。因为凸透镜面积远大于小孔，因此通光量显著增强，成像质量提升。通过透镜组合采集的图像仍可以近似成针孔模型进行分析。



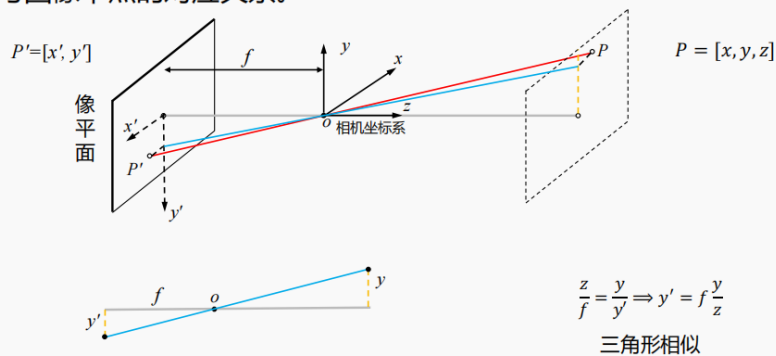
这样的问题在于，成像是倒立的。处理时不方便。因此我们引入“虚拟像平面”。

为了便于研究物体的实际几何信息，在计算机视觉中，通常对与实际物体同方向、与像平面中物体同大小的**虚拟像平面**中的图像进行处理。



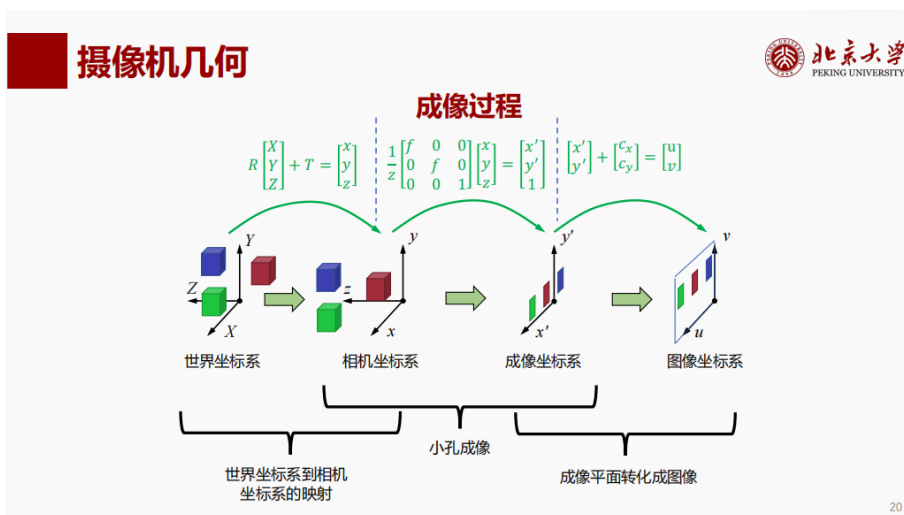
再来看空间坐标（三维）与图像坐标（二维）的关系。

从空间坐标到图像坐标：通过三角形相似关系可以得到空间中的点与图像中点的对应关系。



15

综合来看，坐标变化的全流程是：（其中 $\mathbf{R}$ 是3*3旋转矩阵， $\mathbf{T}$ 是3*1平移矩阵）

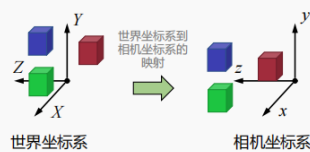


20

第一步中出现了矩阵乘法和加法，我们希望形式统一为乘法，因此我们引入齐次坐标。

成像过程公式化表达

(a) 三维坐标系的转换矩阵



从世界坐标系到相机坐标系的映射过程可以拆解成绕坐标原点**旋转R**和**平移T**两个过程，即： $R \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + T = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ 其中 $T = \begin{bmatrix} d_x \\ d_y \\ d_z \end{bmatrix}$

利用齐次坐标，这两个过程可以统一写成： $\begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$ 或简单写成 $\begin{bmatrix} R & T \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$

因此，世界坐标系中的点 $P = (X, Y, Z)$ 到相机坐标系中点 $P = (x, y, z)$ 的映射过程为 $\begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$

22

第二三步合在一起，引入**内参矩阵**。

摄像机几何

成像过程公式化表达

(d) 成像过程的矩阵表示

对于相机坐标系上一点 $P = (x, y, z)$ ，其在图像坐标系中的坐标为 $P^* = (u, v)$ ，满足公式(2)：

$$v = f \frac{y}{z} + c_y, u = f \frac{x}{z} + c_x$$

为便于 P^* 相关的矩阵计算，引入齐次坐标：

$$\textcircled{1} \text{ 坐标系坐标转化成齐次坐标: } (x, y, z) \Rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad (u, v) \Rightarrow \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

$$\textcircled{2} \text{ 齐次坐标转坐标系坐标: } \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} = \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix}, \text{ 其中 } w \text{ 是齐次项参数。}$$

通常， P 和 P^* 的齐次坐标分别写作：

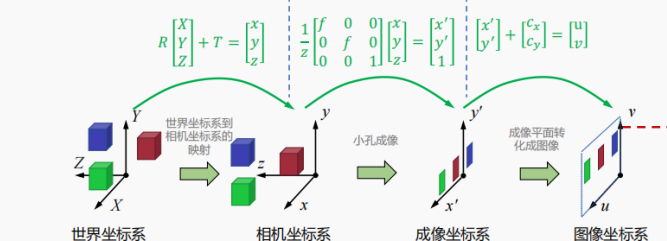
$$P = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad P^* = \begin{bmatrix} f \frac{x}{z} + c_x \\ f \frac{y}{z} + c_y \\ 1 \end{bmatrix} = \frac{1}{z} \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \frac{1}{z} KP \quad \Rightarrow \quad P^* = \begin{bmatrix} u' \\ v' \\ w' \end{bmatrix} = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = KP$$

其中， K 称为相机的内参矩阵

24

摄像机几何

成像过程



对于世界坐标系中的点 (X, Y, Z) ，其通过相机转化为图像上投影点 (u, v) 的过程为：

$$K = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad K [R \ T] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = w \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

K : 相机内参 $[R \ T]$: 相机外参

25

(这里我咋感觉没什么意思... 形式上的化简而已)

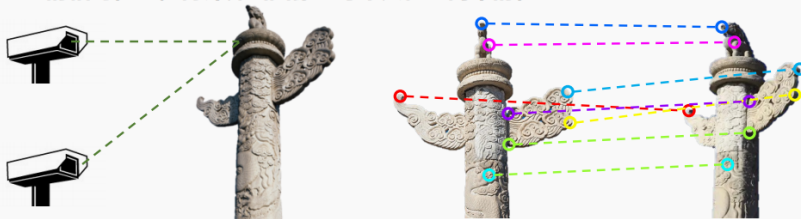
9.2 从二维到三维：三维重建

9.2.1 三维重建基本原理

根据两个相机图像中的对应点关系，先求解两个相机之间的相对位置，再计算出图像上每个点在空间中的位置。

基于对应点重建

三维重建基本原理：根据两个相机图像中的对应点关系，先求解两个相机之间的相对位置，再计算出图像上每个点在空间中的位置



假设两幅图像存在对应点：

$$\begin{aligned} (u_1^1, v_1^1) &\leftrightarrow (u_2^1, v_2^1) \\ (u_1^2, v_1^2) &\leftrightarrow (u_2^2, v_2^2) \\ &\vdots \\ (u_1^n, v_1^n) &\leftrightarrow (u_2^n, v_2^n) \end{aligned}$$

对于世界坐标系中的点 (X, Y, Z) ，其在两相机得到的图像上的投影点计算方式为：

$$A_1 \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = w_1 \begin{bmatrix} u_1 \\ v_1 \\ 1 \end{bmatrix} \quad A_2 \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = w_2 \begin{bmatrix} u_2 \\ v_2 \\ 1 \end{bmatrix}$$

$A_1 = K_1 [R_1 \ T_1]$, $A_2 = K_2 [R_2 \ T_2]$ 分别为两个相机的内参矩阵 K 和世界坐标系到相机坐标系的变换矩阵 $[R \ T]$ (外参) 的乘积，表示从世界坐标系到图像坐标系的变换过程。

w_1 和 w_2 是齐次项参数。

29

9.2.2 基于对应点重建

特征点检测和匹配

常用的特征点描述子

- Harris角点检测
- 高斯-拉普拉斯特征检测子(Laplacian of Gaussian, LoG)
- 尺度不变特征变换(Scale-invariant feature transform, SIFT)
- 加速稳健特征算法(Speeded Up Robust Features, SURF)
- 加速分割测试特征提取(Features From Accelerated Segment Test, FAST)
- 特征点检测算子(Oriented FAST and Rotated BRIEF, ORB)

以SIFT为例

第一，**检测关键点**：找出图像上具有特殊性质的位置。

第二，**生成关键点描述子**：为每个关键点生成一个局部特征向量，用来描述这个点附近长什么样。

第三，**关键点匹配**：比较两张图中关键点描述子的相似度，找到匹配点对

SIFT的优点：对旋转、尺度缩放、亮度变化、视角变化、噪声比较稳定，特征独特性好，适合匹配。

9.3 前沿研究

基于对应点的方式的缺点在于，不能很好地处理遮挡现象、纹理缺乏、光照变化等情况。

10 自然语言处理——句法分析与分词

基础概念：

句法 Syntax 规定了如何用单词组成句子。也就是说，句法关心的是词的排列顺序、短语结构、主谓宾关系是否合法。所以句法判断的核心是这个句子的结构合不合法。

语义 Semantic 语义是语言包含的意义。

10.1 上下文无关文法 CFG

文法 (grammar) 是一组规则，定义了合法短语的树结构。**语言** (language) 是遵循这些规则的句子集。**终结符** (terminal symbols) 是最终出现在句子里的真实单词 (she, saw, city)，**非终结符** (non-terminal symbols) 是中间的语法类别，还可以继续展开 (S, NP, VP, N, V, D)。

比如我们想描述句子 She saw the city.

可以用一些规则：

```
S → NP VP
NP → N
NP → D N
VP → V NP
N → she | city
D → the
V → saw
```

这里：

- **S** 表示 sentence, 句子
- **NP** 表示 noun phrase, 名词短语
- **VP** 表示 verb phrase, 动词短语
- **N** 表示 noun, 名词
- **D** 表示 determiner, 限定词/冠词
- **V** 表示 verb, 动词

于是 **she saw the city** 可以被分析成：

```

S
├─ NP
│  └─ N
│      └─ she
└─ VP
    ├── V
    │   └─ saw
    └─ NP
        ├── D
        │   └─ the
        └─ N
            └─ city
```

也就是说，CFG 不是单纯判断“这句话能不能出现”，而是给出它的层级结构。

为什么叫上下文无关？因为一个非终结符怎么展开，只取决于它自己，不取决于它出现在什么上下文里。

这里的规则从哪里来？人工标注好的语料里学的。

PCFG 给CFG的规则加概率，比如

```
NP → N
NP → D N
```

PCFG会给出每种展开发生的概率

10.2 句法分析 Parsing

已知一套 CFG 规则，给你一句话，问：这句话能不能被这套规则生成？如果能，它的语法树是什么？

自顶向下走的话，缺点是可能乱试并且可能陷入死循环。

CYK算法——自底向上的图表句法分析算法

给定一个句子，先看每个单词能由哪些非终结符生成；再看每两个词、三个词、四个词……能由哪些非终结符生成；最后看整个句子能不能由 **s** 生成。

CYK 处理的 CFG 必须是 **CNF 形式**（乔姆斯基范式）。

CNF:

1. 核心形式只有两种

最重要的是这两种：

$A \rightarrow B C$

或者：

$A \rightarrow a$

其中：

- A, B, C 是非终结符，比如 S, NP, VP, N, V
- a 是终结符，比如真正的单词 $she, saw, city$

所以 CNF 的意思是：

一个非终结符要么推出两个非终结符，要么直接推出一个终结符。

例如下面是 CNF 允许的：

```
S → NP VP
VP → V NP
N → city
V → saw
```

但下面这些不是 CNF：

```
S → NP VP PP    // 右边有三个符号，不行
NP → the N       // 右边混了终结符和非终结符，不行
VP → V           // 右边只有一个非终结符，不符合标准 CNF
```

为什么 CYK 喜欢 CNF？因为 CNF 让句法树变成二叉树结构。同时，任何普通 CFG 都可以转成弱等价的 CNF。

一个例子：

句法分析 (Parsing)

CYK算法 (作者: Ali Cocke, Daniel Younger, Tadeo Kasami)

- CNF语法G:
 - ✓ $S \rightarrow AB \mid BC$
 - ✓ $A \rightarrow BA \mid a$
 - ✓ $B \rightarrow CC \mid b$
 - ✓ $C \rightarrow AB \mid a$
- 输入句子:
 $w = baaba$

步骤2: 填充表格

第五行考虑长度为5的子串 'baaba':
四种组合:
['b', 'aaba'], ['baab', 'a'],
['ba', 'aba'], ['baa', 'ba']

5	{S,A,C}				
4	{∅}	{S,A,C}			
3	{∅}	{B}	{B}		
2	{S,A}	{B}	{S,C}	{S,A}	
1	{B}	{A,C}	{A,C}	{B}	{A,C}
	'b'	'a'	'a'	'b'	'a'

Source: <https://www.educative.io/answers/what-is-the-cyk-algorithm>



北京大学
PEKING UNIVERSITY

左边的数字表示长度， $X(i, j)$ 代表从第 i 个到第 j 个字母形成的子串从最底层开始填表。

最终判断规则：看开始符号 s 是否出现在整个字符串对应的格子 $x(1, n)$ 里。

概率版本的 CYK

普通 CYK 的表格里存的是集合，概率版本则把表格元素从布尔值变成概率值。

10.3 n元模型 n-gram

1-gram = unigram

2-gram = bigram

3-gram = trigram

连续n个词放在一起看，类似于切片？

核心用途：用前面的词预测后面的词。

4. n-gram 的核心用途：用前面的词预测后面的词

n-gram 不只是切片，它通常用来做语言模型。

比如 bigram 模型会估计：

$P(\text{下一个词} \mid \text{前一个词})$

比如：

$P(\text{truth} \mid \text{the})$

$P(\text{city} \mid \text{the})$

$P(\text{room} \mid \text{the})$

trigram 模型会估计：

$P(\text{下一个词} \mid \text{前两个词})$

比如：

$P(\text{truth} \mid \text{be the})$

$P(\text{room} \mid \text{into the})$

这就是一个很朴素的语言模型思想：

根据最近 $n-1$ 个词，预测下一个词。

比如看到：

must be the

后面接 **truth** 的概率可能比较高。



10.4 分词 Tokenization

分词：将文本序列分割为独立的单词。

断句：将文本序列分割成独立的句子。为什么不能简单按句号断句？因为 Mr.Jones 也有句号。

中文分词：难点在于根本没有空格

- 基于字符串匹配的分词方法
拿待分词的字符串，去和一个“充分大的词典”里的词匹配；匹配到了，就认为这是一个词。
- 基于机器学习的分词方法。

10.5 基于马尔科夫模型的文本生成

Markov 链具有无后效性：达到一个状态后，决策只与当前状态有关，而与以前的历史状态无关。

n-gram 语言模型本质上就是一种马尔可夫假设。bigram 对应一阶马尔可夫，它假设下一个词只依赖前一个词。trigram 对应二阶马尔可夫，它假设下一个词只依赖前两个词。

所以 n-gram 的本质是：用有限长度的历史近似完整上下文。

11 自然语言处理——统计语言模型与词表示

11.1 词袋模型 (Bag-of-Words Model, BoW)

概念 不考虑文本中词与词之间的上下文关系，仅仅只考虑所有词的权重，而权重与词在文本中出现的频率有关。以词的统计直方图作为该文本（文档/句子）的特征。

STEPS 数据收集—构建字典—生成特征向量

如果给一句全新的话，其中有单词不在词典里，那我们就直接扔掉不管。然后按照字典顺序统计词频。

缺点 字典可能极大，文本向量稀疏（字典可能很大但是一句话可能就几十个词，算出来的特征向量里大部分位置都是0，浪费计算资源），丢失语序，没有体现关键词的重要性（the, is 出现频率极高）

11.2 朴素贝叶斯 (Naive Bayes)

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}.$$

场景：电商商品评价的情感分类

- 比如："My grandson loved it! So much fun!" 显然是正面评价（好评 😊）。

实操：



朴素贝叶斯 (Naive Bayes) 模型

"My grandson loved it!"

↓

待求解: $P(\text{😊} | \text{"my grandson loved it"})$

不考虑上下文关系 ↑

$P(\text{😊} | \text{"my", "grandson", "loved", "it"})$

||

$$\frac{P(\text{"my", "grandson", "loved", "it"} | \text{😊}) P(\text{😊})}{P(\text{"my", "grandson", "loved", "it"})}$$

两步非常重要的简化

1. 扔掉分母

无论我们算这句话是好评 (😊) 还是差评 (😞), 它们的分母

$P(\text{"my"}, \text{"grandson"}, \text{"loved"}, \text{"it"})$

是一模一样的。因为我们最终只是想比较好评的得分高, 还是差评的得分高, 所以分母完全可以不考虑, 算出来的分子得分直接比较大小即可。

2. “朴素”假设 (词间独立)

分子里的 $P(\text{"my"}, \text{"grandson"}, \text{"loved"}, \text{"it"} | \text{😊})$ 太难算了。要想满足这个条件, 你必须在历史好评里找到这四个词同时出现的句子, 概率极低。

为了解决这个问题, 算法做了一个非常“简单粗暴 (朴素)”的假设: **假设句子里每个词的出现都是绝对独立的, 谁也不影响谁。**

于是, 复杂的联合概率, 被拆成了连乘:

$P(\text{"my"} | \text{😊}) \times P(\text{"grandson"} | \text{😊}) \times P(\text{"loved"} | \text{😊}) \times P(\text{"it"} | \text{😊})$

现在的问题在于, 万一 "grandson" 这个词从来没有出现过, 直接概率为0了, 即使有其他很正面的词汇也不行。为了修正这个错误, 引入**加法平滑**和**拉普拉斯平滑**。

加法平滑 (additive smoothing):

这是最基础的平滑方法。原来的公式是:

$$P(\text{"loved"} | \text{😊}) = \frac{\text{包含 "loved" 的正面样本数量}}{\text{正面样本总词数}}$$

现在, 我们给分子加上一个非常小的常数 α , 同时为了保证概率总和还是1, 分母也要相应地调整。修改后的公式变成

$$P(\text{"loved"} | \text{😊}) = \frac{\text{包含 "loved" 的正面样本数量} + \alpha}{\text{正面样本总词数} + \text{字典总词数} + \alpha}$$

拉普拉斯平滑 其实就是令 $\alpha = 1$ 的加法平滑。

11.3 信息检索 (tf-idf)

什么是好的关键词?

- **词频 (Term Frequency, tf):**

如果一篇文章中某个词反复出现, 那这个词很可能就是这篇文章的重点

- **逆文档频率 (inverse document frequency, idf):**

衡量某个词在语料库的所有文档中的罕见程度。

$$idf = \log_{10} \frac{D}{1+d}$$

其中 D 表示语料库中包含的文档总数量, d 表示语料库中出现某个词的文档数量。

意义:

- 如果一个词在**几乎所有的**文章里都出现了 (比如 "the"), 那它根本不能用来区分这篇文档的主题, 它的信息量极低, 权重应该被调低。
- 如果一个词只在**极少数的**几篇文章里出现 (比如 "holmes" 仅仅在关于福尔摩斯的小说里出现), 那它一旦出现, 就极具代表性, 权重应该被调高。

$$tf-idf = tf \times idf$$

11.4 词表示(Word Representation)

11.4.1 独热表示One-Hot

(这玩意有什么用啊?)

假设我们的字典里只有 4 个词: ["he", "wrote", "a", "book"]。

所谓“独热”(只有一个是热的/亮的), 就是用一个长度为 4 的向量来表示一个词, 并且这个向量里只有对应位置是 1, 其他全是 0。

- he: [1, 0, 0, 0]
- wrote: [0, 1, 0, 0]
- a: [0, 0, 1, 0]
- book: [0, 0, 0, 1]

缺点和词袋模型差不多, 庞大稀疏并且毫无语义关联。

相当于给每个词一个计算机看得懂的身份证?

11.4.2 分布式表示 Distributed Representation

理论基础: 上下文相似的词, 其语义也相似。(词的语义由其上下文决定)

选择一种方式描述上下文, 然后选择一种模型刻画目标词与其上下文之间的关系

11.4.3 Word2Vec

- CBOW: 用上下文预测中心词

• CBOW模型

- ① 当前词的上下文词语的one-hot编码输入到输入层(例如当前词的前后两个词)
- ② 这些词向量分别乘以同一个矩阵 W (周围词向量矩阵) 后分别得到各自的 $1 \times N$ 向量
- ③ 将这些 $1 \times N$ 向量取平均为一个 $1 \times N$ 向量
- ④ 将这个 $1 \times N$ 向量乘矩阵 W' (中心词向量矩阵), 变成一个 $1 \times V$ 向量
- ⑤ 做Softmax分类, 与真实标签 $1 \times V$ 向量计算交叉熵损失
- ⑥ 每次前向传播之后反向传播误差, 调整矩阵 W 和 W' 的值

- Skip-gram

12 自然语言处理——Transformer

13 知识图谱

13.1 概念

知识图谱的概念

- **知识图谱** (Knowledge Graph)本质上是一种大规模**语义网络** (Semantic Network)
- **实体** (entity): 表示客观存在的事物或抽象概念
 - C罗、葡萄牙、皇家马德里
- **关系** (relationships): 表示两个实体 (分为**主实体**和**客实体**) 之间的某种关联
 - 获得奖项、举办单位、效力球队

*图片来源: <https://www.secrss.com/articles/6106>

知识图谱的概念

通常情况下, 知识图谱以**三元组**的形式保存, 即将知识图谱拆分成**<节点, 边, 节点>**, 也称**<实体, 关系, 实体>**。其中, 实体为特定的实例对象, 关系为实体间抽象的关联, 例如<小明, 喜欢, 猫>。

8

13.2 应用

7. 应用场景一: 辅助搜索 —— 语义搜索

这是知识图谱最经典、最成功的应用, 也是 Google 在 2012 年提出知识图谱的**直接目的**。

- **传统搜索的痛点**: 基于“关键词匹配”, 只能返回包含关键词的网页, 无法理解用户查询的真正意图。例如搜索“北京大学的校长是谁”, 传统搜索可能返回大量包含“北京大学”和“校长”的新闻, 但不会直接给出答案。
- **知识图谱的解决方案**: 实现“Things, not Strings” (事物, 而非字符串)。搜索引擎不再处理无意义的字符串, 而是理解查询背后的真实事物和它们之间的关系。
- **效果**: 当你搜索“北京大学创办时间”时, 搜索引擎会直接在结果页顶部显示“1898 年”, 这正是知识图谱中<北京大学, 创办于, 1898年>这个三元组的直接输出。

9. 应用场景三：辅助推荐系统

知识图谱为推荐系统带来了革命性的提升，解决了传统推荐系统 "冷启动" 和 "可解释性差" 两大难题。

- **传统推荐的局限：**基于用户行为的协同过滤，只能推荐 "和你相似的人喜欢的东西"，无法理解内容本身的语义。
- **知识图谱的优势：**
 1. **挖掘更深层次的兴趣：**通过实体关系链进行推荐。例如用户喜欢《阿飞正传》，知识图谱可以发现它的主演是张国荣，进而推荐张国荣主演的《霸王别姬》；再发现《霸王别姬》的导演是陈凯歌，进而推荐陈凯歌的其他电影。
 2. **可解释推荐：**可以告诉用户 "为什么推荐这部电影"，例如 "推荐《宋代皇帝》，因为它和你喜欢的《霸王别姬》都是历史题材"。
 3. **解决冷启动问题：**对于新用户或新物品，没有行为数据时，可以基于知识图谱中的语义关系进行推荐。

Q: 大语言模型也是将用户的自然语言问题转化为对知识图谱的查询语句，然后从图谱中检索答案并返回吗？

A: 一、纯大语言模型：知识存在于 "参数" 中，而非 "图谱" 中

首先要明确：没有外挂任何外部工具的纯大语言模型（比如早期的 GPT-3），它的工作原理和知识图谱问答完全不同。

纯大模型的问答流程

预训练阶段：模型在海量互联网文本上进行训练，将所有学习到的知识（事实、逻辑、语言规律）以权重参数的形式，分布式地存储在神经网络的千亿 / 万亿个参数中。

这是一种隐式的、参数化的知识存储，没有任何明确的 "实体"、"关系" 或 "三元组" 结构。

你无法从模型参数中直接找到 "<北京大学，创办于，1898 年>" 这样一条明确的知识。

推理阶段：当你输入一个问题时，模型会根据它在预训练中学习到的语言模式和统计规律，自回归地一个词一个词地生成答案。

它不会 "查询" 任何外部数据库，也不会 "调用" 任何知识图谱。

它的所有答案，都是基于它 "脑子里记住的" 参数化知识生成的。

纯大模型问答的致命缺陷

这就是为什么大模型会产生 "幻觉"（一本正经地胡说八道）的根本原因：

知识是模糊的、统计性的，而非精确的、事实性的

知识是静态的，截止到预训练数据的时间点，无法获取最新信息

知识是不可控的，无法保证输出的事实准确性

知识是不可解释的，你无法知道它的答案来自哪里

二、传统知识图谱问答：知识存在于 "图谱" 中，而非 "参数" 中

我们之前讲过的传统知识图谱问答系统，工作流程正好和纯大模型相反：

传统知识图谱问答的流程

构建阶段：人类专家或自动化工具将知识以显式的、结构化的三元组形式，存储在知识图谱数据库中。

这是一种符号化的知识存储，每一条知识都是明确、可验证、可追溯的。

你可以直接在数据库中查询到 "<北京大学, 创办于, 1898 年>" 这条记录。

推理阶段:

第一步: 将用户的自然语言问题解析成知识图谱的查询语言 (如 SPARQL)

第二步: 在知识图谱数据库中执行这个查询, 得到精确的结构化结果

第三步: 将结构化结果转换成自然语言答案返回给用户

传统知识图谱问答的致命缺陷

这也是为什么知识图谱问答没有大规模普及的根本原因:

自然语言理解能力弱: 只能处理预设好的简单问题, 无法理解复杂、模糊、口语化的提问

泛化能力差: 只要问题的表达方式稍微变化, 或者知识图谱中没有完全匹配的三元组, 就无法回答

覆盖范围有限: 只能回答知识图谱中存在的事实性问题, 无法进行推理、创作或情感交流

三、当前主流: 大模型 + 知识图谱的融合架构

正是因为纯大模型和纯知识图谱问答各有致命缺陷, 所以现在所有实用的智能问答系统, 都采用了 "大模型 + 知识图谱" 的融合架构。这也是你问题中描述的工作方式。

这种融合架构结合了两者的优点:

用大模型强大的自然语言理解和生成能力, 解决传统知识图谱问答的 "理解难" 和 "表达难" 问题

用知识图谱精确、可验证、可更新的结构化知识, 解决大模型的 "幻觉" 和 "知识过时" 问题

融合架构的两种主流实现方式

方式一: 大模型做 "翻译官", 知识图谱做 "知识库"

这是最接近你问题描述的方式, 也是目前最成熟、应用最广的方式。

完整流程:

用户输入自然语言问题: "北京大学创办于哪一年?"

大模型将自然语言问题翻译成知识图谱的查询语句:

sparql

```
SELECT ?date WHERE {  
  <北京大学> <创办于> ?date .  
}
```

知识图谱数据库执行这个查询, 返回精确结果: 1898年

大模型将结构化结果包装成流畅的自然语言答案: "北京大学创办于 1898 年, 初名京师大学堂。"

优点: 答案 100% 准确, 完全来自知识图谱, 没有幻觉; 知识更新只需要更新知识图谱, 不需要重新训练大模型。

典型应用: 现在的百度、谷歌搜索引擎的 "知识卡片", 银行、运营商的智能客服, 医疗、法律等专业领域的问答系统。

方式二: 大模型做 "生成器", 知识图谱做 "检索器"

这种方式更灵活，适合处理更复杂的问题。

完整流程：

用户输入自然语言问题："请介绍一下北京大学的历史。"

知识图谱检索系统根据问题中的关键词，从知识图谱中检索出所有相关的三元组：

<北京大学, 创办于, 1898年>

<北京大学, 初名, 京师大学堂>

<北京大学, 1912年改为现名, 北京大学>

...

将这些相关知识作为上下文，和用户的问题一起输入给大模型

大模型根据上下文和问题，生成一段流畅、连贯的自然语言介绍

优点：可以处理更复杂、更开放的问题，生成的答案更自然、更全面。

注意：这种方式下，大模型仍然可能产生幻觉，但因为有知识图谱提供的上下文作为约束，幻觉的概率会大大降低。

13.3 知识图谱的构建和应用

13.3.1 知识抽取

1. 实体抽取

从文本中抽取原子信息元素，通常包括人名、组织名、地理位置、时间 / 日期、专有名词等

i. 基于人工设计的模板和规则

例子："认为"、"坐落于"、"战胜****" 这样的模板

ii. 基于序列标注的机器学习方法

将实体识别转化为一个序列标注问题，给文本中的每个词打上标签（如 "人名 - 开始"、"人名 - 中间"、"非实体"）

2. 关系抽取

给定两个实体，确定它们之间存在什么关系。

a. 基于模板的方法

b. 基于监督学习的方法

3. 事件抽取

a. 事件发现和分类：识别文本中是否发生了事件，并判断事件的类型。

b. 事件要素提取：提取出事件的各个要素，包括触发词、时间、地点、参与者、结果等

13.3.2 知识表示

(Knowledge Representation, KR) 就是用易于计算机处理的方式来描述人脑的知识，并支持通过符号之间的运算来模拟人脑的推理过程。

1. 符号表示法 (传统):

a. 逻辑表示法 (一阶谓词逻辑)

分为：个体词、谓词、量词、逻辑联结词

谓词逻辑不仅能表示知识，还能进行精确的推理。

注意： $P=F, Q=T \rightarrow P \rightarrow Q=T$ ：前提不成立，承诺未被打破，为真。事实上， $P=F$ 时无论 Q 是什么， $P \rightarrow Q$ 都是 T 。

e.g.

前提 1：北京大学在且只在青年节举办校庆

$(\forall t)(Youth(t) \Leftrightarrow Anniversary(PKU, t))$

前提 2：北京大学今天举办校庆

$Anniversary(PKU, Today)$

结论：今天是青年节

$Youth(Today)$

b. 产生式表示法 (三部分组成)

i. 规则库：存储所有的产生式规则，格式为 IF 条件 THEN 结论

- 例子：IF X是大学 THEN X是高等学校

ii. 综合数据库：存储当前已知的事实

- 例子：北京大学是大学

iii. 推理机：负责匹配规则和事实，推导出新的结论

- 例子：事实北京大学是大学 + 规则 IF X是大学 THEN X是高等学校 $\rightarrow$ 结论北京大学是高等学校

c. 框架表示法

框架表示法

框架(Frame)是描述对象（事物，事件或概念）属性的一种数据结构。**框架网络(Frame Network)**是由不同的框架通过属性之间的关系而建立起来的联系，从而形成的网络结构。框架网络能够充分表达相关对象之间的各种关系。

例1：教师的类框架
框架名：<教师>
年龄：(1~120)
工作：范围：(教学，科研)
缺省：教学
性别：(男，女)
学历：(学士，硕士，博士)
类型：(<小学教师>，<中学教师>，<大学教师>)

74

- <人>是<教师>的父类

- <大学教师>是<教师>的子类

子类可以继承父类的所有属性，这大大减少了知识的冗余。

2. 向量表示法（现代）：

前面三种都是基于离散符号的表示法，它们在处理小规模、确定性知识时表现很好，但在互联网时代的海量、模糊、动态知识面前，显得力不从心。

向量表示法的出现，彻底改变了这一局面。它的核心思想是：将知识图谱中的每个实体和关系，都映射到一个低维、连续的向量空间中，用向量来表示它们的语义。

基于离散符号的知识表示	基于连续向量的知识表示
显式知识、强逻辑约束	隐式知识、弱逻辑约束
可解释性强	不易解释
推理不易扩展	可对接神经网络、易于扩展
表示空间小	表示空间巨大

向量表示法的模型与训练：

22.2 核心思想：基于平移的模型（TransE）

这是最经典、最有影响力的知识图谱嵌入模型，它的灵感来自于一个有趣的语言学现象：

在词向量空间中，语义相近的词会聚集在一起，并且具有相同关系的词对之间的向量差近似相等。

最著名的例子就是：

向量(国王) - 向量(王后) $\approx$ 向量(男人) - 向量(女人)

TransE 把这个思想推广到了知识图谱中，提出了一个非常简单但极其有效的假设：

对于知识图谱中的任意一个三元组 $\langle h, r, t \rangle$ （头实体，关系，尾实体），都满足：

$$\mathbf{h} + \mathbf{r} \approx \mathbf{t}$$

也就是说，关系 r 可以表示为头实体 h 到尾实体 t 的一个平移向量。

知识图谱的向量表示法（知识嵌入），本质上是 NLP 语义向量（词嵌入）思想在图结构数据上的扩展和升级。

所有语义向量技术，无论是 NLP 中的词嵌入（Word Embedding），还是知识图谱中的实

体 / 关系

嵌入，都建立在同一个伟大的理论假设之上：**分布语义假设 (Distributional Hypothesis)**。**分布语义假设**：一个词的含义，由它所处的上下文环境决定。在相似上下文中出现的词，具有相似的语义。

这个假设是现代自然语言处理的基石，也是所有向量表示法的灵魂。它告诉我们：不需要人工定义词的含义，只需要统计词在海量文本中出现的上下文，就可以用一个向量来表示词的语义。向量之间的距离，就代表了语义之间的相似度。无论是 Word2Vec、BERT 的词向量，还是 TransE 的实体 / 关系向量，本质上都是在实践这个假设。

13.3.3 知识推理

1. 逻辑表示推理

推理规则:逻辑等价式

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

其中:

① $P \rightarrow Q$ 等价于 $\neg P \vee Q$;

③ $\neg(P \vee Q)$ 等价于 $\neg P \wedge \neg Q$;

⑤ $R \wedge (P \vee Q)$ 等价于 $(R \wedge P) \vee (R \wedge Q)$;

② $P \leftrightarrow Q$ 等价于 $(P \wedge Q) \vee (\neg P \wedge \neg Q)$

④ $\neg(P \wedge Q)$ 等价于 $\neg P \vee \neg Q$

⑥ $R \vee (P \wedge Q)$ 等价于 $(R \vee P) \wedge (R \vee Q)$

(看PPT吧...)

13.4 图神经网络

为什么需要GNN?

图数据是一种完全不同的数据类型，它有自己独特的特点，传统神经网络无法处理。

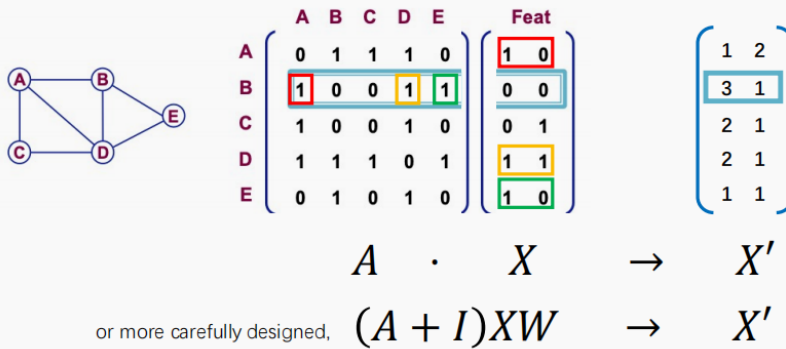
图神经网络的核心思想：**通过聚合邻居节点的特征，学习中心节点的表示。**

你是谁，取决于你和谁交朋友；要了解一个节点，最好的方法就是了解它的所有邻居，把邻居的特征收集起来（聚合），就能得到这个节点自己的特征。

图神经网络(GNN)



一种最简单的实现:



四、最简单的 GNN 实现

PPT 第 56 页给出了一个最简单的 GNN 的数学公式，我来逐部分给你解释：

$$(A + I) X W = X'$$

这个公式看起来复杂，其实拆解开来非常简单：

- **X**：节点特征矩阵，每一行是一个节点的特征向量
- **A**：邻接矩阵， $A[i][j]=1$ 表示节点 i 和节点 j 之间有边，0 表示没有边
- **I**：单位矩阵，对角线是 1，其他是 0。加上 I 的意思是：**不仅聚合邻居的特征，还要保留自己的特征**
- **W**：可学习的权重矩阵，这是整个模型中唯一需要训练的参数
- **X'**：输出的新的节点特征矩阵，每一行是 GNN 学习到的节点表示

整个公式的含义：

1. 用邻接矩阵 A 乘以节点特征 X ，得到每个节点的邻居特征的总和
2. 加上自己的特征（乘以 I ）
3. 乘以一个可学习的权重矩阵 W ，进行特征变换
4. 得到新的节点表示 X'

这就是一个最简单的 GNN 层的全部计算过程。

14 智能机器人

15 强化学习

强化学习是智能体通过与环境交互，以最大化长期累积奖励期望为目标，学习最优行为策略的机器学习范式，核心思想来自“强化理论”：通过奖励与惩罚塑造行为。

智能体 $\rightarrow$ 观察状态 $s \rightarrow$ 执行动作 $a \rightarrow$ 环境状态变为 $s' \rightarrow$ 获得奖励 $r \rightarrow$ 智能体更新策略

15.1 马尔可夫决策过程 (MDP)

马尔可夫性：未来的状态仅由**当前状态和行动**决定，与过去的所有历史无关。

MDP 五元组： $M=(S,A,P,R,\gamma)$

策略：

策略是智能体在任意状态下选择动作的规则，是强化学习最终要学习的目标。

- **确定性策略：** $a=\pi(s)$ ，一个状态对应唯一动作
- **随机策略：** $a\sim\pi(a|s)$ ，一个状态对应动作的概率分布，满足 $\sum_a\pi(a|s)=1$

轨迹与累积收益：

轨迹 (Trajectory/Episode/Rollout)： 智能体执行策略产生的完整交互序列

$\tau=(s_0,a_0,r_0,s_1,a_1,r_1,\dots,s_T,r_T)$

累积收益 (回报 Return)： 轨迹的总奖励，引入折扣因子保证无限步收益有限

$G(\tau)=r_0+\gamma r_1+\gamma^2 r_2+\dots=\sum_{i=0}^T \gamma^i r_i$

- $\gamma=1$ ：远近收益同等重要
- $\gamma=0$ ：只关心即时奖励（完全贪心）
- 常用取值：0.9~0.99

累积收益 (回报)

$$G(\tau)=r_0+\gamma r_1+\gamma^2 r_2+\dots=\sum_{i=0}^T \gamma^i r_i$$

确定性策略

$$a=\pi(s)$$

随机策略归一化条件

$$\sum_a \pi(a|s)=1, \quad \forall s \in S$$

状态价值函数

$$V_{\pi}(s)=E_{\tau \sim \pi}[G(\tau)|s_0=s]=E_{\tau \sim \pi}[\sum_t \gamma^t r_t|s_0=s]$$

动作价值函数 (Q 函数)

$$Q_{\pi}(s,a)=E_{\tau \sim \pi}[G(\tau)|s_0=s,a_0=a]$$

价值函数关系

□ 确定性策略： $V_{\pi}(s)=Q_{\pi}(s,\pi(s))$

○ 随机策略： $V_{\pi}(s)=\sum_a \pi(a|s) Q_{\pi}(s,a)$

贝尔曼期望方程（状态价值）

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V_{\pi}(s')]$$

贝尔曼期望方程（动作价值）

$$Q_{\pi}(s,a) = \sum_{s',r} p(s',r|s,a)[r + \gamma V_{\pi}(s')]$$

贝尔曼最优方程（状态价值）

$$V(s) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$$

贝尔曼最优方程（动作价值）

$$Q(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \max_{a'} Q(s',a') \right]$$